LECTURE NOTES · PRETRAINING

Scaling Laws, Visually

Companion notes to the slides

How loss falls with parameters, data and compute; the history before language models; the key papers and results; and how to design, run and read a scaling experiment. One section per slide, with every term defined and the intuition spelled out.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Scaling Laws, Visually* (31 slides) slide by slide. Numbers quoted from a paper were checked against that paper; numbers marked “derived” were computed from quoted ones; curves the slides label “illustrative” are sketches of a mechanism, not data. The $1M worked example is fictional.

Contents

How to use these notes **3**

1 Overview (slides 1–3) **3**
1.1 Slide 1: Title: Scaling Laws, Visually . . . 3
1.2 Slide 2: What a scaling law is, and why anyone cares . . . 4
1.3 Slide 3: Reading a log-log plot . . . 5

2 History before language models (slides 4–6) **6**
2.1 Slide 4: History I: learning curves, 1990s . . . 6
2.2 Slide 5: History II: more data beats cleverer algorithms, 2001–2017 . . . 7
2.3 Slide 6: History III: deep learning scaling is predictable, 2017–2019 . . . 8

3 Kaplan et al. 2020 (slides 7–8) **9**
3.1 Slide 7: Kaplan et al. 2020: three laws for language models . . . 9
3.2 Slide 8: Kaplan's recipe: grow the model faster than the data . . . 10

4 Chinchilla (slides 9–13) **12**
4.1 Slide 9: Chinchilla, 2022: three ways to find the compute-optimal split . . . 12
4.2 Slide 10: Chinchilla's result: scale parameters and tokens together . . . 13
4.3 Slide 11: Why Kaplan and Chinchilla disagreed . . . 14
4.4 Slide 12: The parametric form, term by term . . . 15
4.5 Slide 13: Why power laws at all? . . . 17

5 Refinements (slides 14–18) **18**
5.1 Slide 14: Refinement I: the cost of serving changes the optimum . . . 18
5.2 Slide 15: Refinement II: when data runs out, repeat it . . . 19
5.3 Slide 16: Refinement III: hyperparameters have scaling laws too . . . 20
5.4 Slide 17: Refinement IV: the schedule decides what a point is worth . . . 21
5.5 Slide 18: Refinement V: laws for architecture knobs . . . 22

6 Beyond loss (slides 19–20) **24**
6.1 Slide 19: From loss to capability: what scaling laws do not promise . . . 24
6.2 Slide 20: Predicting a benchmark score: the two-stage map . . . 25

7 Ladders (slides 21–23) **26**
7.1 Slide 21: The ladder: one recipe, several sizes, one fit . . . 26
7.2 Slide 22: Designing a ladder: the choices and the usual answers . . . 27
7.3 Slide 23: Fitting the law and reading the forecast . . . 28

8 Practice (slides 24–25) **29**
8.1 Slide 24: Worked example: a $1M scaling study . . . 29
8.2 Slide 25: Pitfalls: how scaling experiments go wrong . . . 30

9 Frontier (slides 26–29) 31
9.1 Slide 26: The frontier: where the laws are being pushed 31
9.2 Slide 27: Case study: the MAI-Base-2 ladder (2026) 32
9.3 Slide 28: Exercises 33
9.4 Slide 29: Worked answers to exercises 1–3 34

10 Wrap-up (slides 30–31) 35
10.1 Slide 30: Summary: what to remember 35
10.2 Slide 31: Sources 36

A Notation 39

B Formulas at a glance 40

Index of terms 41

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the equations written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Two conventions. Logarithms written log are base 10 unless a base is shown; natural logarithms are written ln. Loss is measured in nats per token unless stated otherwise (one nat is $1/\ln 2 \approx 1.44$ bits). The symbols used throughout are collected in the notation table in Appendix A, and the main formulas on one page in Appendix B.

A reading plan for a single lecture: Sections 1 and 2 are background and can be assigned as reading; Sections 3 and 4 (Kaplan, Chinchilla) are the core and take about half the session; Section 5 (refinements) can be sampled; Sections 7 and 8 (ladders, practice) are what students should be able to do afterwards, and the exercises in Section 10 test exactly that.

1 Overview (slides 1–3)

1.1 Slide 1: Title: Scaling Laws, Visually

How loss falls with parameters, data and compute; the history before language models; the key results; and how to design, run and read a scaling experiment of your own.

What the slide shows A log-log sketch: six measured points on a straight line ("measured ladder"), a dashed continuation, and a star labelled "forecast" well beyond the last point. The whole lecture is in this picture. A family of models is trained at increasing sizes; their losses fall on a line; the line is extended to a size nobody has trained yet; the star is the prediction that the big run then has to confirm.

Details The deck is organized as ten footer stages: Overview, History, Kaplan, Chinchilla, Refinements, Beyond loss, Ladders, Practice, Frontier, Wrap-up. The first six are about *what is known*: the empirical regularities and the papers that established them. The last four are about *what to do*: how to run the experiment yourself, what can go wrong, and where the method is being pushed now. The scope is pretraining only: the loss of a model trained from scratch on a large corpus to predict the next token. Fine-tuning, reinforcement learning and inference-time compute are mentioned only where they change what a pretraining law can promise (slides 19 and 26).

Terms

pretraining The first and largest phase of training a language model: next-token prediction on a broad corpus, from random initialization, measured by the cross-entropy loss on held-out text. Everything after it (instruction tuning, preference optimization, reasoning training) is *post-training*.

scaling law An empirical relationship, usually a power law, between a model family's loss and a resource spent on it (parameters, data or compute), holding the rest of the training recipe fixed. "Law" is used in the physicist's sense of a regularity that holds over a wide range, not a theorem.

ladder A set of models trained with one fixed recipe at several sizes, so that the only variable is scale. The rungs are the sizes; the fit through them is the law; the top of the ladder is the model you intend to build. Slides 21–23 are about ladders.

forecast A prediction of the loss (or a derived quantity) of a run that has not been done, obtained by extrapolating a fitted law.

Intuition

A scaling law is a measuring instrument, not a theory of intelligence. Its value is that it converts a question nobody can answer by thinking ("how good will a model 100 times bigger be?") into a question that can be answered by a few cheap experiments and a ruler. Everything else in the lecture is about how to use the ruler without fooling yourself.

1.2 Slide 2: What a scaling law is, and why anyone cares

An empirical regularity: a model family's loss falls as a power of the resource spent, smoothly, over many decades.

What the slide shows On the left, loss against training compute on log-log axes from 10^{17} to 10^{24} FLOPs: five points on a straight line with the equation $L = AC^{-\alpha}$. On the right, the three resources N, D, C with one-line descriptions, and a box with the three uses of a law: forecast, budget, compare.

Details The defining feature is the straight line on log-log axes. If $L = AC^{-\alpha}$ then $\log L = \log A - \alpha \log C$: a line with slope $-\alpha$. The slope is small for language models ($\alpha \approx 0.05$ for the compute law in Kaplan et al.), which means that each tenfold increase in compute lowers the loss by only a factor $10^{-0.05} \approx 0.89$, about 11%. That sounds like little, but it is reliable: the same 11% per decade, decade after decade, over the seven orders of magnitude Kaplan et al. measured. The reliability, not the size, is what makes the law useful.

The three resources are linked by the cost model $C \approx 6ND$ (slide 10 derives it), so only two of them are free. Most of the lecture is about how to split a compute budget C between N and D, and what loss to expect from the split.

The three uses map onto three sections of the deck. *Forecast* (slides 21–23): extrapolate the line. *Budget* (slides 9–10, 14): at fixed C, find the (N, D) that minimizes L. *Compare* (slides 23, 25): two recipes give two lines; the horizontal gap between them, read in compute, is a far more stable number than the vertical gap read in loss at one size.

Terms

parameters (N) The number of trainable weights in the model. Conventions differ on whether the embedding matrices are included; Kaplan et al. exclude them ("non-embedding parameters"), Hoffmann et al. include them. The convention matters at small scale, where embeddings are a large fraction (slide 11).

tokens (D) The number of training tokens seen, counting repeats. A token is a subword unit from the tokenizer's vocabulary; roughly three quarters of an English word.

compute (C) The total number of floating-point operations spent in training, in FLOPs. Often expressed as *PF-days* (10^{15} FLOP/s for one day, 8.64×10^{19} FLOPs) or GPU-hours.

FLOPs Floating-point operations (plural of FLOP). One multiply-add counts as two FLOPs. Not to be confused with FLOP/s, a rate.

loss (L) The cross-entropy of the model's next-token distribution against the actual next token, averaged

over held-out tokens, in nats per token. Lower is better. It is the quantity every law in this lecture predicts.

cross-entropy For a true next token x and model probability $p(x)$, the cross-entropy is $-\ln p(x)$; averaged over a text it measures how many nats the model needs per token to encode the text. Its minimum over all models is the entropy of the text.

nat The unit of information when logarithms are natural; 1 nat $= 1/\ln 2 \approx 1.443$ bits.

power law A relationship of the form $y = Ax^{-\alpha}$. Its signature is a straight line on log-log axes with slope $-\alpha$. Contrast an exponential, $y = Ae^{-x/\lambda}$, which is a straight line on log-linear axes.

exponent (alpha) The power in a power law. For scaling laws it is dimensionless and typically between 0.05 and 0.4; it says how fast returns diminish.

Intuition

Think of the exponent as an exchange rate between compute and quality. With $\alpha = 0.05$, "10× more compute" and "11% less loss" are the same thing, and a recipe change worth 11% less loss is worth a tenfold compute saving. Learning to move between the two currencies is the skill the lecture teaches, and slide 23 shows why the conversion is less stable than it looks.

1.3 Slide 3: Reading a log-log plot

A pure power law is a line whose slope is the exponent. A floor bends it, and subtracting the floor straightens it again.

What the slide shows Three panels. Left: two pure power laws, $\alpha = 0.1$ and $\alpha = 0.2$, both straight, the steeper one with the larger exponent. Middle: $L = E + Ax^{-\alpha}$ with a floor $E = 2$; the curve bends and flattens toward the dashed line at E. Right: the same data plotted as $L - E$; straight again.

Details Three constants describe a law with a floor:

$$L(x) = E + Ax^{-\alpha}, \quad \text{with } E \text{ the floor (irreducible loss), } A \text{ the scale, } \alpha \text{ the exponent.}$$

Only the *excess* loss $L - E$ is a power law. On a log-log plot of L itself, the curve is straight while $Ax^{-\alpha} \gg E$ and bends as the excess becomes comparable to the floor. Fitting a straight line through a bent region gives an exponent that is too small and a forecast that is too pessimistic (the line flattens early); fitting with a floor that is too large gives the opposite error. Since E is the least well determined of the three constants (it is set by the behavior at the far end of the data, where the points are fewest and most expensive), long extrapolations are sensitive to it. Slide 23 makes this quantitative.

Curvature has three usual causes: a floor; a change of regime (for instance, from being limited by parameters to being limited by data; slide 13); or a mistuned point (a rung whose learning rate or schedule was wrong; slides 16–17). Before fitting, look at the residuals of a straight-line fit: a systematic bow is a sign that one of the three is present.

Terms

log-log plot A plot with logarithmic scales on both axes. Equal distances are equal *ratios*, so a decade (factor of 10) is the same width everywhere, and a power law is a straight line.

floor (E) Also *irreducible loss*. The loss that remains as the resource goes to infinity. For language modeling it is the entropy of the text itself: the unpredictability of language that no model can remove.

irreducible error The same idea for a classification error rate: the Bayes error, below which no classifier can go because the labels themselves are noisy or ambiguous.

residual The difference between an observed value and the fitted curve's prediction at that point. Residuals that wander systematically (all positive, then all negative) mean the functional form is wrong.

decade A factor of ten. "Two decades of compute" means a factor of 100.

Intuition

The exponent is a slope and the floor is an asymptote; you cannot read either off a plot by eye when they are both present, because a floor makes a straight line look like it is losing slope. The practical rule: always plot $L - E$ for the E you are proposing, and only believe the fit if what you see is straight.

2 History before language models (slides 4–6)

2.1 Slide 4: History I: learning curves, 1990s

Long before transformers, statisticians fit power laws to error versus training-set size and extrapolated them.

What the slide shows A learning curve in the classical sense: test error falling and training error rising with the number of training examples n, both approaching the same asymptote a from opposite sides. The dashed line at a is labelled "predicted from the small-n points".

Details *Cortes, Jackel, Solla, Vapnik and Denker* (NIPS 1993) proposed modeling the two curves jointly as

$$E_{\text{test}}(n) \approx a + \frac{b}{n^{\alpha}}, \qquad E_{\text{train}}(n) \approx a - \frac{c}{n^{\alpha}},$$

fitting a, b, c and α on the small-n points available, and using the fitted a to compare classifiers without training any of them to convergence. Their motivation was practical: in 1993 a full training run on the largest available data could take weeks, and they wanted to choose among architectures early. That is the ladder idea thirty years before the word was used for language models.

Amari (1992–93) and others in the statistical-mechanics-of-learning tradition derived the asymptotic form of such curves for idealized learners: for a model that can represent the target function exactly, the generalization error falls like $1/n$ (exponent 1), with constants set by the number of parameters. *Vapnik*'s uniform-convergence theory gives distribution-free *upper bounds* on the generalization gap of order $\sqrt{h/n}$, where h is the VC dimension of the model class. Bounds of this type are worst-case over all data distributions and are far looser than what real curves show; the empirical exponents of the next two slides are the gap between the two.

Two differences from the modern setting are worth noting. First, these curves are in the number of examples n at fixed model; the model-size axis appears only with Rosenfeld et al. (slide 6). Second, they measure error rates, which have a Bayes floor, where modern laws measure cross-entropy, which has an entropy floor; the mathematics is the same.

Terms

learning curve Error (or loss) as a function of the number of training examples, at fixed model and training procedure. The modern "scaling law in D" is a learning curve.

asymptotic error (a) The limit of the learning curve as $n \to \infty$; the error the model would reach with

unlimited data.

generalization error Expected error on new examples from the same distribution; estimated by the error on a held-out test set.

VC dimension (h) A combinatorial measure of a model class's capacity: the largest number of points it can label in every possible way. Appears in distribution-free bounds on the generalization gap.

uniform convergence The property that empirical error is close to true error *simultaneously for every model in the class*, which is what makes a worst-case bound possible. The price is that the bound must hold for the worst distribution too.

Intuition

Test and training error pinch together from opposite sides, and the place they meet is the quantity you want. Fitting both curves is a way of triangulating the asymptote. The lesson that survives into 2026: a few cheap, well-chosen runs can tell you where an expensive run will end up, provided you have the right functional form.

2.2 Slide 5: History II: more data beats cleverer algorithms, 2001–2017

In NLP and then vision, accuracy kept rising log-linearly with data over three decades, for every learner tried.

What the slide shows Accuracy against training-set size in words, 10^5 to 10^9, for three learners A, B, C. All three rise roughly linearly in $\log n$; their order changes along the way; none flattens. The curve is illustrative, after Banko and Brill (2001).

Details *Banko and Brill* (ACL 2001) studied confusion-set disambiguation: choosing between easily confused words ({to, too, two}, {principal, principle}) from context. Their training corpus went up to one billion words, three orders of magnitude more than the largest previously used for the task. Their learners were winnow, a perceptron, naive Bayes and a simple memory-based (nearest-neighbor) learner. All four kept improving, approximately log-linearly, out to a billion words, and the ranking among them changed with corpus size: the learners that won on small data were not the ones that won on large data. Their conclusion was that effort spent on collecting data would pay better than effort spent on algorithms, a view that was contrarian at the time.

Halevy, Norvig and Pereira (IEEE Intelligent Systems, 2009) restated this for the web era in "The Unreasonable Effectiveness of Data": simple models with access to very large corpora (their examples were from machine translation and speech) outperform elaborate hand-built models trained on small data.

Sun, Shrivastava, Singh and Gupta (ICCV 2017) tested the same question in vision with JFT-300M, an internal Google dataset of 300 million images with 375 million noisy labels. Performance on downstream tasks (classification, detection, segmentation) rose logarithmically with the amount of pretraining data, with no sign of saturation at 300 million, and the gains were larger for larger backbone networks: capacity has to grow with data to keep the curve going.

A caution about "log-linear": accuracy rising linearly in $\log n$ is not a power law in accuracy, but it is consistent with a power law in *error* over a modest range (since $1 - \text{err}$ with a small-exponent power law looks nearly log-linear). These papers did not fit exponents; they established that the curves do not flatten within reach.

Terms

confusion-set disambiguation An NLP task: given a sentence with a blank and a small set of commonly confused words, pick the right one from context. It needs only unlabeled text as training data, which is why corpus size could be scaled so far.

log-linear Growing linearly in the logarithm of the input: equal increments for every tenfold increase. A straight line on a plot with log x-axis and linear y-axis.

winnow, perceptron, naive Bayes Three classical linear or probabilistic classifiers: winnow and the perceptron are online algorithms that learn a weight vector by multiplicative and additive updates respectively; naive Bayes assumes features are conditionally independent given the class.

memory-based learner A nearest-neighbor method: store the training examples and label a new one by the labels of its closest stored examples.

JFT-300M Google's internal image dataset of about 300 million images with noisy, automatically derived labels, used to study vision scaling before public datasets of that size existed.

Intuition

> The ranking of methods is a function of data size. A method that wins at 10^5 examples has no claim to winning at 10^9. This is why scaling experiments compare *curves*, not points (slide 21): the only fair comparison between two recipes is at the scale you intend to use.

2.3 Slide 6: History III: deep learning scaling is predictable, 2017–2019

Hestness et al. measured learning curves across domains: the same three-region shape, with task-specific exponents.

What the slide shows Generalization error against training-set size on log-log axes: flat at small n ("small-data region"), a straight descent with slope β ("power-law region"), and a bend toward a dashed floor ("irreducible error"). The sketch follows Hestness et al. (2017).

Details *Hestness et al.* (Baidu, 2017, "Deep Learning Scaling is Predictable, Empirically") trained families of deep networks on four kinds of task while varying training-set size over two or more decades, re-tuning model size and hyperparameters at each point. They found the same three regions everywhere: a small-data region where the model is no better than guessing, a power-law region, and a region dominated by the irreducible error. The exponents they report for the power-law region (generalization error $\propto n^{\beta_g}$) are:

Task	β_g
Neural machine translation	-0.128
Word-level language modeling (LSTM; RHN)	-0.066; -0.070
Character-level language modeling	-0.094 (SGD); -0.095 (Adam)
Image classification, ImageNet top-1 (top-5)	-0.309 (-0.488)
Speech recognition (DS2; attention model)	-0.299; -0.296

Two observations from the paper shaped everything that followed. First, the exponent depends on the task and data, not on the architecture: within a task, changing the architecture or optimizer shifted the curve up or down (a better model starts lower) but did not change its slope. Second, the exponents are far from the -0.5 that simple theory suggests for the data axis, in both directions, so they must be measured rather than assumed.

Rosenfeld, Rosenfeld, Belinkov and Shavit (2019, "A Constructive Prediction of the Generalization Error Across Scales") then proposed a single functional form in both the data size n and the model size m, fit it on

small configurations and extrapolated to larger ones, for vision and language tasks. This is the first explicit joint law $L(n,m)$ and the direct ancestor of Kaplan's $L(N,D)$ and Chinchilla's parametric fit.

Amodei and Hernandez (OpenAI blog, 2018, "AI and Compute") estimated that the compute used in the largest training runs had grown by about 300,000× between 2012 and 2018, doubling every 3.4 months. Once compute was growing that fast, the question of what each doubling buys became the central planning question in the field.

Terms

small-data region The range of n where the model has not learned anything beyond the prior; error is flat near its maximum. Its extent depends on the task.

power-law region The range where error is a power law in n; the regime scaling laws are fit in.

Bayes error The lowest achievable error rate on a task, set by the overlap of the classes under the true distribution; the floor of the error learning curve.

LSTM, RHN Two recurrent architectures for sequence modeling (long short-term memory; recurrent highway network), the state of the art for language modeling in 2017, before the transformer.

Deep Speech 2 (DS2) Baidu's end-to-end speech recognition model, one of the two speech architectures in Hestness et al.

Intuition

Architecture moves the curve; data sets its slope. If that is true, then once you have a reasonable architecture the questions worth asking are about data and scale, and architecture search is a search for a lower intercept, not a steeper line. Most later evidence supports this for the dense transformer; slides 13 and 18 give the caveats.

3 Kaplan et al. 2020 (slides 7–8)

3.1 Slide 7: Kaplan et al. 2020: three laws for language models

Loss is a power law in each of parameters, data and compute when the other two are not the bottleneck.

What the slide shows Three log-log panels, each a straight line drawn from the paper's fitted constants: loss against non-embedding parameters N (10^5 to 10^9), against tokens D (10^7 to 10^{11}), and against compute $C_{\min}$ in PF-days (10^{-6} to 10). The exponents are printed on each panel.

Details *Kaplan, McCandlish, Henighan, Brown, Chess, Child, Gray, Radford, Wu and Amodei* (OpenAI, January 2020, "Scaling Laws for Neural Language Models") trained decoder-only transformers on WebText2 across a wide range of sizes and reported three single-variable laws, each valid when the other two resources are not limiting:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \alpha_N \approx 0.076, \qquad N_c \approx 8.8 \times 10^{13} \text{ (non-embedding parameters)}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \qquad \alpha_D \approx 0.095, \qquad D_c \approx 5.4 \times 10^{13} \text{ tokens}$$

$$L(C_{\min}) = \left(\frac{C_c}{C_{\min}}\right)^{\alpha_C^{\min}}, \qquad \alpha_C^{\min} \approx 0.050, \qquad C_c \approx 3.1 \times 10^8 \text{ PF-days}$$

Note that these are written *without* a floor: on the range measured, the data were consistent with a pure power law, and the floor was not resolved. The constants N_c, D_c, C_c are not meaningful sizes (they are the values at which the fitted loss would reach 1 nat); they simply set the scale of the line.

Three findings beyond the laws themselves. (1) *Shape barely matters.* At fixed N, varying depth, width, the number of heads or the feed-forward ratio over a wide range changes loss by a few percent; N changes it by the law. This is what licenses the ladder construction of slide 10 in Part Two, where one scalar sets the whole architecture. (2) *The joint law.* With both N and D finite,

$$L(N,D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}, \quad \text{fitted with } \alpha_N = 0.076,\ \alpha_D = 0.103,\ N_c = 6.4\times 10^{13},\ D_c = 1.8\times 10^{13}.$$

The joint fit has its own constants (slightly different from the single-variable ones). It implies that larger models are more sample-efficient (reach a given loss in fewer tokens) and that overfitting sets in when $N^{0.74}/D$ grows, so that to avoid it the data should grow at least like $N^{0.74}$. (3) *Seven decades.* Some of the trends span more than seven orders of magnitude in compute, which is the empirical basis for trusting extrapolation at all.

The compute law uses C_{min}, the compute a run *would* take at the critical batch size (slide 8); runs at larger batch waste compute on steps that are too small to help, and the paper corrects for this so that the compute axis reflects an efficient run.

Terms

non-embedding parameters All weights except the token embedding (and, with tied weights, the output) matrix. Kaplan et al. found the laws cleaner in this count, because embeddings are a large fraction of small models but contribute little to the loss trend.

PF-day One petaFLOP/s sustained for one day: $10^{15} \times 86{,}400 = 8.64 \times 10^{19}$ FLOPs. GPT-3's training compute of about 3.1×10^{23} FLOPs is roughly 3,600 PF-days.

C-min (compute at the critical batch) Kaplan's notation for the compute a training run would require if run at the critical batch size; the compute law is stated in C_{min} so that it describes compute-efficient training rather than whatever batch size was used.

sample efficiency How much loss improvement a model extracts per training token. Larger models are more sample-efficient: at equal tokens their loss is lower, and they reach a target loss with fewer tokens.

overfitting (in scaling) The regime where loss stops improving with N because D is too small for the model; Kaplan's joint law places its onset where the term D_c/D becomes comparable to $(N_c/N)^{\alpha_N/\alpha_D}$.

WebText2 OpenAI's web-scraped training corpus (an extension of the GPT-2 corpus), on which the 2020 laws were measured. Exponents are specific to the data distribution.

Intuition

Each law answers a different question. $L(N)$: if data were free, how much does a bigger model help? $L(D)$: if the model were huge, how much does more data help? $L(C)$: with both chosen well, what does compute buy? The exponent on compute is the smallest of the three because compute must be split between the other two; neither resource can be grown at the full rate.

3.2 Slide 8: Kaplan's recipe: grow the model faster than the data

Allocating a growing budget by the 2020 fits sends most of each new decade of compute to parameters, not tokens.

What the slide shows Two bar pairs showing what 10× more compute buys under each rule: Kaplan 2020 ($N \propto C^{0.73}$, $D \propto C^{0.27}$) multiplies the model by 5.4 and the data by 1.9; Chinchilla 2022 ($N \propto C^{0.5}$, $D \propto C^{0.5}$) multiplies both by 3.2. Side boxes: GPT-3 as the recipe in practice, and the critical batch size.

Details From the compute law and the joint law, Kaplan et al. derived how an optimal run should grow with the budget: $N \propto C^{0.73}$, batch size $B \propto C^{0.24}$ and steps $S \propto C^{0.03}$, hence tokens $D = BS \propto C^{0.27}$. Since $0.73 + 0.27 = 1$, the two rules multiply back to the compute budget (with $C \propto ND$). For a tenfold budget: $10^{0.73} = 5.4\times$ the parameters and $10^{0.27} = 1.9\times$ the tokens.

Two further conclusions followed. *Larger models are more sample-efficient*, so for a fixed loss target the cheapest route is a model well above the size that would converge within the budget. And *the compute-efficient frontier stops short of convergence*: the best use of a budget is to train a big model and stop while its loss is still falling fast. The slogan "train big, stop early" summarizes both.

The recipe was followed. GPT-3 (May 2020) has 175B parameters and was trained on 300B tokens, about 1.7 tokens per parameter; Gopher (DeepMind, December 2021) has 280B parameters on 300B tokens. Under the Chinchilla analysis two years later (slides 9–10) both were far from compute-optimal: at their budgets, a model four times smaller trained on four times more tokens would have had lower loss.

Critical batch size. Kaplan et al. also fit

$$B_{\text{crit}}(L) = \frac{B_*}{L^{1/\alpha_B}}, \qquad B_* \approx 2 \times 10^8 \text{ tokens}, \qquad \alpha_B \approx 0.21,$$

building on *McCandlish, Kaplan and Amodei* (2018, "An Empirical Model of Large-Batch Training"). Below B_{crit}, doubling the batch halves the number of steps needed (perfect data parallelism); above it, extra batch is wasted because the gradient estimate is already accurate. The critical size depends on the loss, not on the model: as training proceeds and loss falls, the gradient signal gets smaller relative to its noise and larger batches become useful. This is why batch size is increased during a run and why Kaplan's C_{min} correction is needed to compare runs fairly.

Terms

compute-efficient frontier For each compute budget, the lowest loss any (N, D) achieves; equivalently, the lower envelope of all training curves plotted against compute. Kaplan's $L(C_{\text{min}})$ is a fit to this frontier.

batch size (B) The number of tokens (or sequences) whose gradients are averaged in one optimizer step. Tokens D, steps S and batch B satisfy $D = BS$.

critical batch size The batch size at which the trade-off between steps and examples turns: below it, bigger batches mean proportionally fewer steps at the same tokens; above it, they mostly cost extra tokens. McCandlish et al. relate it to the *gradient noise scale*, the ratio of gradient variance to squared gradient norm.

gradient noise scale $B_{\text{noise}} = \text{tr}(\Sigma)/|G|^2$, where G is the true gradient and Σ the per-example gradient covariance. It predicts the critical batch size: when the batch is much smaller than B_{noise} the update is noise-dominated and averaging more examples helps linearly.

convergence The point in training where loss stops improving with more steps at fixed data; Kaplan's recipe deliberately stops well before it.

GPT-3, Gopher OpenAI's 175B-parameter model (2020) and DeepMind's 280B-parameter model (2021), both trained on about 300B tokens following the 2020 allocation.

Intuition

The 2020 picture: parameters are the scarce ingredient and data is cheap, so pour each new unit of compute mostly into the model. It was internally consistent and it produced GPT-3. Its error was not in the data but in how the small runs were read (slide 11), and correcting that error moves the split from 73/27 to 50/50.

4 Chinchilla (slides 9–13)

4.1 Slide 9: Chinchilla, 2022: three ways to find the compute-optimal split

Over 400 models from 70M to 16B parameters, one question: at a fixed compute budget, how big should the model be?

What the slide shows An illustrative IsoFLOP figure: three U-shaped curves of final loss against model size N on a log axis, one per compute budget (10^{20}, 10^{21}, 10^{22} FLOPs); each curve's minimum is marked and the minima are joined by a dashed line, the trace of $N_{\text{opt}}(C)$. On the right, the three approaches numbered 1 to 3.

Details *Hoffmann et al.* (DeepMind, March 2022, "Training Compute-Optimal Large Language Models") trained more than 400 transformers from 70 million to over 16 billion parameters on 5 to 500 billion tokens, with the learning-rate schedule matched to each run's length (slide 11), and asked a single question: given a fixed training compute C, what N and D minimize the final loss? They answered it three ways.

1. **Fix N, vary D.** For each model size, train on several token counts, and from each training curve record the loss at every compute value along the way. The lower envelope across all sizes is the best achievable loss at each C; the (N, D) that achieves it, plotted against C, gives $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$. Result: $a = 0.50$, $b = 0.50$.
2. **IsoFLOP profiles.** Fix C at one of nine budgets; train models of different N, each on exactly $D = C/(6N)$ tokens; plot final loss against $\log N$. The result is a valley, since a model that is too small cannot use the tokens and one that is too large sees too few of them. Fit a parabola, take its minimum, repeat for each budget. Result: $a = 0.49$, $b = 0.51$.
3. **Parametric fit.** Assume $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$, fit all runs at once, then minimize analytically subject to $6ND = C$ (slide 12). Result: $a = 0.46$, $b = 0.54$.

All three give $a \approx b \approx 0.5$: parameters and tokens should grow at the same rate. The agreement between three methods with different assumptions is the evidence; no one of them is conclusive alone, and slide 11 shows that the third one's published constants are in fact the least reliable of the three.

Terms

compute-optimal The (N, D) pair that minimizes loss at a fixed training compute budget. "Chinchilla-optimal" is this pair under Hoffmann et al.'s fits: roughly 20 tokens per parameter.

IsoFLOP profile A set of runs with the same compute budget but different model sizes (and hence different token counts), plotted as loss against size. "Iso" means equal.

lower envelope The pointwise minimum of a family of curves; here, the best loss attained at each

compute by any run.

parametric fit Fitting an assumed closed-form function (here, with five constants) to all the data at once, as opposed to reading off minima nonparametrically.

final loss The loss at the end of a training run whose learning-rate schedule has fully decayed, as opposed to an intermediate loss read off a longer run (slide 17).

Intuition

The IsoFLOP valley is the clearest picture. Walk along a line of constant compute: on the left a tiny model trained on an ocean of tokens it cannot absorb; on the right a huge model that has seen too few tokens to learn anything; the best point is in the middle, and the three approaches are three ways of locating it. As the budget grows the valley moves right, and the question is how fast: Kaplan said $C^{0.73}$, Chinchilla says $C^{0.5}$.

4.2 Slide 10: Chinchilla's result: scale parameters and tokens together

About twenty tokens per parameter at the compute-optimal point, and a 70B model that beat a 280B one at equal cost.

What the slide shows A table comparing Gopher (280B parameters, 300B tokens, 1.1 tokens per parameter) with Chinchilla (70B, 1.4T, 20 tokens per parameter) at the same compute, 5.8×10^{23} FLOPs; the rules $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$, $C \approx 6ND$; and the rule of thumb $N \approx \sqrt{C/120}$, $D \approx 20N$. Side boxes derive $6ND$ and interpret the exponents.

Details Hoffmann et al. tested their prediction by training Chinchilla, a 70B-parameter model, on 1.4 trillion tokens: the same budget as Gopher, four times fewer parameters, 4.7 times more tokens. Chinchilla outperformed Gopher on nearly every evaluation and, being four times smaller, is four times cheaper to run at inference, which is why the result changed practice immediately. (By the $6ND$ approximation Chinchilla cost 5.9×10^{23} and Gopher 5.0×10^{23} FLOPs; the paper's own accounting, which includes attention and embedding operations, puts both at 5.76×10^{23}. The approximation is within 15%.)

The rule of thumb. With $C = 6ND$ and $D = 20N$: $C = 120N^2$, so

$$N \approx \sqrt{C/120}, \qquad D \approx 20N, \qquad \text{e.g. } C = 10^{22} \text{ FLOPs} \Rightarrow N \approx 9\text{B}, D \approx 180\text{B}.$$

Tenfold compute gives $\sqrt{10} = 3.2\times$ the parameters and $3.2\times$ the tokens.

Where $6ND$ comes from. For a dense transformer, each parameter participates in one multiply-add per token in the forward pass (2 FLOPs), and the backward pass costs about twice the forward (one matrix product to get gradients with respect to the activations, one to get gradients with respect to the weights): $2N + 4N = 6N$ FLOPs per token, times D tokens. The attention score computation adds a term proportional to context length times d_{model} per token, which is negligible unless the context is very long, and the embedding lookups cost nothing. More precise counts (DeepSeek LLM's $M = 72 n_{\text{layer}} d^2 + 12 n_{\text{layer}} d\, l_{\text{seq}}$ FLOPs per token, slide 16) differ at the few-percent level for normal context lengths.

What the exponents mean. The three approaches gave $a = 0.50, 0.49, 0.46$ and $b = 0.50, 0.51, 0.54$. Equal exponents mean that at the optimum a marginal parameter and a marginal twenty tokens are worth the same loss reduction; if one were worth more, the optimum would move toward it. The number 20 is an empirical constant of this data and recipe. Better data lowers the loss at every (N, D) and can move the ratio; sparse models (slide 18) change what "parameter" means; and the refit in slide 12 shows how sensitive it is to fitting choices.

Terms

tokens per parameter (TPP) The ratio D/N. Chinchilla-optimal training has $D/N \approx 20$. Values far above 20 are called *over-training* (slide 14); GPT-3's 1.7 is far below.

Chinchilla DeepMind's 70B-parameter model trained on 1.4T tokens (2022), named after the paper's thesis that a smaller, longer-trained model beats a larger one at equal compute.

6ND The standard estimate of training FLOPs for a dense transformer with N parameters on D tokens: 2 FLOPs per parameter per token forward, 4 backward.

dense model A model in which every parameter is used for every token, as opposed to a mixture-of-experts model where only a fraction is active per token (slides 18, 26).

inference cost The compute to generate with a trained model: about $2N$ FLOPs per token produced, since only the forward pass runs. It scales with N and not with D, which is the starting point of slide 14.

Intuition

At the optimum the two excess-loss terms are in balance: spend a little more on parameters and the data term starts to dominate; spend more on tokens and the capacity term does. "Twenty tokens per parameter" is where that balance sat for Hoffmann et al.'s data and recipe. The exact number is less important than the principle that there is a balance, and that GPT-3-era models were far from it.

4.3 Slide 11: Why Kaplan and Chinchilla disagreed

Same question, exponents of 0.73 versus 0.5. The gap is methodology, and later papers pinned down each piece.

What the slide shows Left: two training curves against the fraction of the token budget seen, a cosine schedule matched to the horizon (green) and one three times too long (orange, dashed); at the horizon the long schedule reads higher. Right: four causes, each attributed to the paper that identified it: learning-rate schedule (Hoffmann 2022), what counts as N (Porian 2024), warm-up and tuning (Porian 2024), fit hygiene (Besiroglu 2024).

Details The two papers asked the same question and got $a = 0.73$ versus $a \approx 0.5$. The difference turned out to be entirely methodological, and resolving it took two years and three further papers.

Learning-rate schedule (Hoffmann et al. 2022, Section 2). Kaplan et al. used a fixed number of training tokens and a fixed cosine schedule for all models, and read intermediate losses off those long runs to populate the small-D points. Hoffmann et al. observed that for a cosine schedule set for 130B tokens, the intermediate loss estimates are overestimates of what a model trained with a matched, shorter schedule reaches: the learning rate has not decayed at the point you read, so the loss is still high. Using the intermediate values underestimates how effective it is to train models on less data, which biases the fit toward larger N. Hoffmann et al. also noted a scale difference: most of their models exceed 500M parameters, while many of Kaplan's were below 100M.

What counts as N (Porian et al. 2024). Porian, Wortsman, Jitsev, Schmidt and Carmon ("Resolving Discrepancies in Compute-Optimal Scaling of Language Models") reproduced both results and found three factors that account for the discrepancy: the computational cost of the last layer (output embedding), which Kaplan excluded by counting non-embedding parameters and which is a large share of compute for small models; the warm-up duration, which Kaplan held constant and which is too long for small models; and scale-dependent optimizer tuning, meaning that learning rate, batch size and AdamW's β_2 must be set separately for each model size. With all three corrected, Kaplan-style data give $a = 0.497$, in excellent

agreement with Hoffmann. They also found that careful learning-rate *decay* is not essential to the agreement, so Hoffmann's own explanation was at most part of the story.

Fit hygiene (Besiroglu et al. 2024). Besiroglu, Erdil, Barnett and You ("Chinchilla Scaling: A replication attempt") extracted the data from Hoffmann's Figure 4 and refit the parametric form. They found that Hoffmann's published approach-3 constants ($E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$) are inconsistent with the paper's approaches 1 and 2, fit the extracted data poorly, and come with implausibly narrow confidence intervals; those constants imply about 70 tokens per parameter rather than 20. Their refit ($E = 1.817$, $A = 482$, $B = 2085$, $\alpha = 0.348$, $\beta = 0.366$, giving $a \approx 0.513$) restores the optimal ratio to around 20 tokens per parameter, with a confidence interval of roughly 4 to 40. The headline result survived; the published constants did not.

Terms

cosine schedule A learning-rate schedule that, after warm-up, decays from its peak to a small fraction along a half-cosine over the planned number of steps. Its final value depends on the planned length, so a run must know its horizon in advance.

warm-up The initial phase of training during which the learning rate ramps up from near zero to its peak, typically over a few hundred to a few thousand steps. Too long a warm-up wastes steps for small models.

horizon The planned total number of steps or tokens of a run, which the schedule is tied to.

intermediate loss A loss reading taken part-way through a run whose schedule continues beyond it. Not equivalent to the final loss of a shorter run.

AdamW beta2 The decay rate of the second-moment (squared-gradient) running average in the Adam/AdamW optimizer; Porian et al. found it must be tuned with scale for small models.

confidence interval A range of parameter values consistent with the data at a stated probability; Besiroglu et al. judged Hoffmann's intervals too narrow to be credible.

Intuition

Every one of the four errors has the same shape: a choice that is fine for the big models is wrong for the small ones, and since the small ones are most of the ladder, they tilt the fit. The practical section of the lecture (slides 22–25) is this list turned into a checklist: tune every rung, end every schedule where you read it, count parameters consistently, and distrust a fit you cannot reproduce by a second method.

4.4 Slide 12: The parametric form, term by term

$L(N,D) = E + A/N^{\alpha} + B/D^{\beta}$ is the workhorse. Each term has a meaning, and the fit has a standard procedure.

What the slide shows Four stacked bars, one per model size $N = 10^8$ to 10^{11}, each at $D = 20N$, split into the floor E (grey), the capacity term A/N^{α} (blue) and the data term B/D^{β} (gold), using Hoffmann's constants. The totals fall from 3.49 to 1.91; the floor is constant at 1.69; both excess terms shrink. Side boxes: the meaning of each term and the fitting procedure.

Details The form

$$L(N,D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

is additive: the loss is a floor plus two independent penalties. E is the entropy of the text under the ideal model. A/N^{α} is the *approximation* penalty: the loss a model of finite capacity would have even with infinite data. B/D^{β} is the *estimation* penalty: the loss a model of infinite capacity would have after seeing only D tokens. The assumption that the two add, rather than interact, is a modeling choice that works well empirically and makes the optimization tractable.

Compute-optimal allocation. Minimize L subject to $6ND = C$. Setting the derivatives along the constraint to zero gives the balance condition and the solution:

$$\alpha \frac{A}{N^{\alpha}} = \beta \frac{B}{D^{\beta}} \text{ at the optimum;} \qquad N_{\text{opt}} = G\left(\frac{C}{6}\right)^{a}, \quad D_{\text{opt}} = G^{-1}\left(\frac{C}{6}\right)^{b}, \quad a = \frac{\beta}{\alpha+\beta}, \quad b = \frac{\alpha}{\alpha+\beta},$$
$$G = \left(\frac{\alpha A}{\beta B}\right)^{1/(\alpha+\beta)}.$$

With Hoffmann's constants, $a = 0.28/0.62 = 0.452$ and $b = 0.548$, which are the approach-3 exponents on slide 10; $G \approx 1.35$. Evaluating at Chinchilla's budget gives $N_{\text{opt}} \approx 32$B and $D_{\text{opt}} \approx 3.0$T, about 90 tokens per parameter (derived), which is the inconsistency Besiroglu et al. flagged: the published constants do not reproduce the paper's own 20. With their refit constants the same formulas give $N_{\text{opt}} \approx 72$B, $D_{\text{opt}} \approx 1.3$T and about 18 tokens per parameter, consistent with the model that was actually trained.

Fitting procedure (Hoffmann et al., Appendix D). Minimize the Huber loss (with $\delta = 10^{-3}$) between $\log \hat{L}$ and $\log L$ over all runs, using L-BFGS started from a grid of initial values for $(\log A, \log B, \log E, \alpha, \beta)$, and keep the best local minimum. The log transform makes the fit relative rather than absolute, so small-loss runs are not swamped by large-loss ones; the Huber loss limits the influence of outlier runs. Besiroglu et al. point out that the choice of δ and the initialization grid affect the result, which is part of why the published constants were off.

Terms

approximation error The part of the loss due to the model class being unable to represent the target perfectly; shrinks with capacity N.

estimation error The part of the loss due to having only a finite sample to fit from; shrinks with data D.

Lagrangian / balance condition The condition for an optimum under a constraint: the marginal benefit per unit of compute must be equal for both uses. Here it reads $\alpha A N^{-\alpha} = \beta B D^{-\beta}$, i.e. the two penalties stand in the fixed ratio β/α at the optimum.

Huber loss A robust fitting loss: quadratic for residuals smaller than δ, linear beyond, so that outliers pull on the fit less than under least squares.

L-BFGS A quasi-Newton optimizer for smooth low-dimensional problems; standard for fitting a handful of constants. It finds a local minimum, which is why one starts it from many points.

local minimum A point where the fitting loss cannot be decreased by small moves but may not be the global best; multiple starts guard against stopping at one.

Intuition

Read the bars from left to right: the floor never moves, the blue capacity term shrinks as N grows, the gold data term shrinks as D grows with it. The optimum is where you cannot shrink either faster by borrowing compute from the other. The exponents α and β decide the exchange rate; if $\beta > \alpha$ (as in the refit), data pays slightly better and $a > 0.5$.

4.5 Slide 13: Why power laws at all?

Three pictures that produce them, each saying what the exponent depends on. None is settled; all are useful.

What the slide shows Three cards: *Resolution on a manifold* (Sharma and Kaplan 2020; Bahri et al. 2021), *Zipf-distributed features* (Hutter 2021), *Quanta of skill* (Michaud et al. 2023), and a note on what the theories agree on.

Details Empirically, power laws are found; theoretically, several mechanisms produce them, and each predicts something about the exponent.

Resolution on a manifold. Sharma and Kaplan (2020, "A Neural Scaling Law from the Dimension of the Data Manifold") argue that natural data lie near a low-dimensional manifold of intrinsic dimension d inside the high-dimensional input space. A network with N parameters can resolve the manifold to a grid spacing that shrinks like $N^{-1/d}$ (it takes $N \propto s^{-d}$ cells to cover a d-dimensional set at spacing s), and for a smooth target function the interpolation error at spacing s scales like s^4 for cross-entropy and squared losses, giving $L - E \propto N^{-4/d}$: the prediction $\alpha \approx 4/d$. A small exponent such as Kaplan's 0.076 then corresponds to $d \approx 50$, which they test against estimates of the intrinsic dimension. Bahri, Dyer, Kaplan, Lee and Sharma (2021, "Explaining Neural Scaling Laws") generalize this and distinguish two regimes: *resolution-limited*, when both N and D are large and the exponent is set by d; and *variance-limited*, when one resource is far larger than the other, in which the exponent is 1 in the smaller resource (loss $\propto 1/N$ or $1/D$, like a sampling variance).

Zipf-distributed features. Hutter (2021, "Learning Curve Theory") studies a toy model in which the data contain many features whose frequencies follow a power law (a Zipf distribution), and each feature is learned once enough examples of it have been seen. The loss remaining after D examples is the total frequency of features not yet seen often enough, which is itself a power law in D with exponent set by the Zipf slope. The model is deliberately minimal and shows that no sophistication in the learner is needed for a power law; the heavy-tailed data distribution is enough.

Quanta of skill. Michaud, Liu, Girit and Tegmark (2023, "The Quantization Model of Neural Scaling") propose that a model's loss is a sum over many discrete skills ("quanta"), each learned abruptly once the model is large enough or has seen enough data, with the skills used in Zipfian frequency. Summing many sharp steps with power-law weights gives a smooth power law in aggregate, which reconciles smooth loss curves with the abrupt per-task behavior of slide 19.

What they agree on. In all three pictures the exponent is a property of the data distribution (its intrinsic dimension, its feature frequencies, its skill frequencies) and not of the architecture, which is consistent with Hestness's observation that architecture shifts curves without changing slopes. It also implies that a new data mixture needs its own ladder: the exponent fit on one corpus is not transferable to another.

Terms

data manifold The hypothesis that high-dimensional natural data (images, text embeddings) are concentrated near a smooth surface of much lower dimension d; d is the *intrinsic dimension*.

intrinsic dimension (d) The number of coordinates needed to describe the data locally; estimated, for example, from nearest-neighbor distances. Sharma and Kaplan relate $\alpha \approx 4/d$.

resolution-limited vs variance-limited Bahri et al.'s two regimes: when the smaller resource is comparable to the larger, loss is limited by how finely the model resolves the data (exponent $4/d$); when one resource dwarfs the other, loss is limited by sampling variance (exponent 1).

Zipf distribution A power-law frequency distribution: the k-th most common item has frequency $\propto k^{-s}$. Word frequencies in text follow it with $s \approx 1$.

quanta (of skill) Michaud et al.'s term for discrete units of capability, each acquired abruptly; a loss curve is the Zipf-weighted sum of many such acquisitions.

Intuition

Three different stories give the same line, which is reassuring about the line and discouraging about identifying the story. The common thread is heavy tails: data in which there is always a rarer feature, a finer detail, a less common skill still to learn. As long as the tail goes on, each decade of resources buys the next slice of it, and that is a power law.

5 Refinements (slides 14–18)

5.1 Slide 14: Refinement I: the cost of serving changes the optimum

Chinchilla minimizes training compute. If the model will answer billions of queries, serving cost belongs in the objective.

What the slide shows Loss against tokens per parameter at fixed training compute, drawn with Hoffmann's fitted form: a shallow valley with its minimum near 20 ("Chinchilla-optimal") and a second point at 200, ten times over-trained, with $+0.09$ loss and a $3.2\times$ smaller model. Side box: inference-aware scaling (Sardana and Frankle), over-training laws (Gadre et al.), and practice (Llama 3 8B).

Details The Chinchilla objective is training compute alone. A deployed model is also run for inference, at about $2N$ FLOPs per generated token (slide 10), and a popular model generates far more tokens in its lifetime than it was trained on. *Sardana and Frankle* (2023, "Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws") add this to the objective:

$$C_{\text{total}}(N,D) = \underbrace{6ND}_{\text{training}} + \underbrace{2ND_{\text{inf}}}_{\text{inference}},$$

minimize over (N,D) at a target loss, for an expected lifetime output of D_{inf} tokens.

Since the inference term grows with N only, any nonzero D_{inf} shifts the optimum toward smaller N and larger D; for models expected to serve many tokens, the shift is large and the optimal ratio is far past 20 tokens per parameter. The loss valley of the slide shows why the shift is affordable: at fixed training compute, moving from 20 to 200 tokens per parameter raises loss by only about 0.09 nats under Hoffmann's form, while the model shrinks by $\sqrt{10} \approx 3.2$ and so does every future inference bill.

Over-training is predictable. Gadre et al. (2024, "Language models scale reliably with over-training and on downstream tasks") built a testbed of 104 models from 11M to 6.9B parameters on three data distributions, fit scaling laws that extrapolate in both model size and the amount of over-training, and used them to predict the validation loss of a 1.4B-parameter run on 900B tokens (32 times over-trained relative to Chinchilla, about 640 tokens per parameter) and of a compute-optimal 6.9B run, each from experiments using 300 times less compute. They also proposed a power law relating perplexity to downstream top-1 error and used it to predict that error with 20 times less compute. The over-trained regime is as lawful as the compute-optimal one.

In practice. Llama 3 8B was trained on over 15 trillion tokens, about 1,900 tokens per parameter; frontier models routinely train at hundreds of tokens per parameter. The reason is economic rather than scientific: serving dominates total cost, and a smaller model at slightly higher loss is the cheaper product.

Terms

over-training Training a model on more tokens per parameter than the compute-optimal ratio (roughly, more than 20). It costs training compute for a small loss gain but yields a smaller model.

inference-aware scaling Choosing (N, D) to minimize training plus expected inference compute, rather than training compute alone.

lifetime tokens (D inf) The total number of tokens a deployed model is expected to generate; the quantity that multiplies $2N$ in the inference term.

perplexity e^L for a loss L in nats per token: the effective number of equally likely next tokens the model is choosing among. Gadre et al. state their downstream law in perplexity.

top-1 error The fraction of multiple-choice or classification items where the model's single highest-scoring answer is wrong.

Intuition

Chinchilla answers "cheapest model to *train* to this loss." Deployment asks "cheapest model to *own* at this loss." The second question always prefers a smaller model, and the valley is shallow enough that the preference costs little loss. Twenty tokens per parameter is the floor of sensible ratios, not the target.

5.2 Slide 15: Refinement II: when data runs out, repeat it

Unique text is finite. Muennighoff et al. measured what a repeated token is worth and fit a law with a decay constant.

What the slide shows Effective data D'/U_D against epochs of the same data, drawn from the paper's fitted $R_D^* \approx 15.4$: a curve that starts along the diagonal (every token as good as new), bends away, and saturates near 16. A dashed vertical at 4 epochs marks the region the paper calls nearly free.

Details *Muennighoff et al.* (2023, "Scaling Data-Constrained Language Models") asked what happens to Chinchilla's law when unique tokens U_D are limited and the rest of the budget is spent on repeats. They trained models at many combinations of unique data, epochs and size, and fit an extension of the parametric form in which repeated tokens count as a decaying fraction of fresh ones:

$$D' = U_D + U_D R_D^* \left(1 - e^{-R_D/R_D^*}\right), \quad R_D^* \approx 15.39; \qquad N' = U_N + U_N R_N^* \left(1 - e^{-R_N/R_N^*}\right), \quad R_N^* \approx 5.31;$$

then $L(N', D')$ takes the Chinchilla form. Here R_D is the number of *extra* passes (epochs minus one), D' is the effective number of fresh tokens the repeated data are worth, and the second formula does the same for parameters (U_N is the model size that would be compute-optimal for U_D unique tokens; extra parameters beyond it have decaying value). The headline finding: training for up to about four epochs of repeated data changes loss negligibly compared with having unique data; with more repetition the value of adding compute decays toward zero.

The formula's shape explains the four-epoch rule. For small R_D, $1 - e^{-R_D/R^*} \approx R_D/R^*$ and $D' \approx U_D(1 + R_D)$: every pass is worth a full pass. The marginal value of the k-th extra pass is e^{-k/R^*}, so with $R^* \approx 15$ the fourth pass is still worth $e^{-4/15} \approx 77\%$ of a fresh one, the tenth about half, and the total saturates at $U_D(1 + R^*) \approx 16 U_D$. Repetition is therefore a real regime with a budget rule of its own: when data is the constraint, the surplus compute should go into more epochs and a somewhat larger model, both with decaying returns, rather than being left unspent.

Why it matters now. Villalobos et al. (2022, revised 2024, "Will we run out of data?") project that training datasets will reach the size of the public human-generated text stock between 2026 and 2032, earlier if models

are over-trained. Frontier runs already train on 15–50 trillion tokens; for MAI-Base-2 the 50T-token run was eight passes over 6.25T unique tokens (Part Two), and its ladder repeated data by the same factor on every rung so that the small runs would overfit the way the big one does.

Terms

epoch One full pass over the training data. Language models were traditionally trained for well under one epoch; "four epochs" means each token was seen four times.

unique tokens (U D) The number of distinct tokens available, as opposed to the number seen counting repeats ($D = U_D \times$epochs).

effective data (D') The number of fresh tokens that a repeated corpus is worth, under the fitted decay.

decay constant (R star) The number of extra passes over which the marginal value of a repeat falls by a factor e. Muennighoff et al.'s fitted values: 15.39 for data, 5.31 for parameters.

data-constrained regime Training where the binding limit is unique data rather than compute, so that compute can only be spent on repeats or on a larger model.

source scaling Microsoft AI's term for holding the repetition factor fixed across a ladder, so each rung sees its data as many times as the full run will (Part Two, slide 12).

Intuition

A repeated token is not worthless and not free; it is worth a fraction that shrinks with each pass, and the fraction is well measured. The practical reading: a corpus can be stretched by about four times at almost no cost, by about ten with a visible but tolerable cost, and beyond that you are paying compute for very little.

5.3 Slide 16: Refinement III: hyperparameters have scaling laws too

A ladder is only as good as the tuning of its rungs. Learning rate and batch size move with scale in predictable ways.

What the slide shows Two small log-log panels from DeepSeek LLM's fits: optimal learning rate falling with compute, $\eta_{\text{opt}} = 0.3118\,C^{-0.125}$, and optimal batch size rising, $B_{\text{opt}} = 0.2920\,C^{0.3271}$. Side box: three ways to keep rungs tuned.

Details Slide 11 established that an untuned rung biases the fit. The learning rate and batch size that minimize loss are themselves smooth functions of scale, so they can be fit and extrapolated like everything else.

Sweep and fit. *DeepSeek-AI* (2024, "DeepSeek LLM: Scaling Open-Source Language Models with Longtermism") ran grids of small models over learning rate and batch size, found that near-optimal configurations occupy a broad basin (values within a factor of about two of the best are nearly as good), and fit the basin's center as a power law in compute:

$\eta_{\text{opt}} = 0.3118\,C^{-0.1250}$, $B_{\text{opt}} = 0.2920\,C^{0.3271}$ (tokens), with $C = M\,D$ and M the non-embedding FLOPs per token.

They replaced $6N$ by the more exact per-token count $M = 72\,n_{\text{layer}} d^2_{\text{model}} + 12\,n_{\text{layer}} d_{\text{model}} l_{\text{seq}}$, which includes attention and excludes embeddings. MAI-Base-2 (Part Two, slide 17) fit similar laws in terms of training steps and tokens per parameter rather than compute. The learning-rate exponent is negative (bigger

runs want smaller steps) and the batch exponent positive (bigger runs can average more examples per step), as the critical-batch-size picture of slide 8 predicts.

Parametrize so the optimum does not move. Yang et al. (2022, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer") introduce μP ("maximal update parametrization"): a prescription for how initialization variances and per-layer learning rates should scale with width so that the training dynamics of each layer stay in the same regime as the network widens. Under μP the optimal learning rate found on a narrow model transfers to a wide one ("μTransfer"), which removes the width dimension from the sweep. It does not by itself transfer across depth or across token counts, so it is used together with, not instead of, fitted laws.

Scale the batch, not the steps. From McCandlish et al. (2018): below the critical batch size, doubling the batch halves the steps at the same token count, so the batch can be raised to the critical size at no cost in tokens; above it, additional batch mostly costs extra tokens. Since the critical size grows as loss falls, large runs raise the batch over the course of training. Rules of thumb for the learning rate when the batch changes: linear scaling ($\eta \propto B$) for SGD in the small-batch regime, square-root scaling ($\eta \propto \sqrt{B}$) for Adam-type optimizers (Malladi et al. 2022).

Terms

learning rate (eta) The step-size multiplier on the (preconditioned) gradient in each optimizer update. The single most important hyperparameter; its optimum shrinks with scale.

hyperparameter basin The region of hyperparameter space within which loss is close to its minimum. A wide basin makes tuning forgiving; DeepSeek LLM report a factor of about two in both learning rate and batch.

mu P (maximal update parametrization) A way of scaling initialization and per-layer learning rates with width so that feature learning stays width-independent; its practical payoff is that hyperparameters tuned on a small model transfer to a wide one.

zero-shot hyperparameter transfer Using the hyperparameters tuned on a proxy model directly on the target without re-tuning; the promise of μP across width.

non-embedding FLOPs per token (M) DeepSeek LLM's exact count of the floating-point operations per training token excluding embeddings: $72\,n_{\text{layer}}d^2 + 12\,n_{\text{layer}}d\,l_{\text{seq}}$, used in place of $6N$.

linear and square-root scaling rules Heuristics for adjusting the learning rate when the batch size changes by a factor k: multiply η by k (SGD, small batches) or by $\sqrt{k}$ (Adam-type optimizers).

Intuition

Every rung of a ladder is its own small optimization problem, and the answer drifts with scale in a direction you can predict: smaller steps, bigger batches. If you copy the smallest rung's settings upward, the large rungs are mistuned by a known amount and your law inherits the error. Fit the drift and you remove it; parametrize with μP and part of the drift disappears by construction.

5.4 Slide 17: Refinement IV: the schedule decides what a point is worth

Loss at the end of a cosine run is not the loss at the same token count inside a longer run. Ladders must control for it.

What the slide shows Three loss curves against training steps as a percentage of a long run: a cosine schedule to 100% (orange), a cosine to 50% (green), and a constant learning rate with a decay over the last 10% (blue, WSD), plus a dashed decay branch off the constant run at 40–50%. At the 50% mark, the long

cosine run sits visibly above the run whose schedule ended there: same tokens, higher loss, because the long run has not yet decayed.

Details With a cosine schedule the learning rate at any point depends on the planned total length, so a run's loss at 50% of a long horizon is not the loss it would have reached if 50% had been the horizon. The decayed run has had its learning rate taken to near zero, which lets the weights settle into a lower-loss region; the undecayed run is still taking large steps. Reading intermediate points off a long run therefore *overstates* the loss at smaller token counts, which is the error Hoffmann attributed to Kaplan (slide 11).

One run per point. The safe default is to give each (N, D) point its own run with the schedule ending at D. It is expensive: k token budgets per model size means k full runs, and most of the compute goes into the longest ones.

Warmup-stable-decay (WSD). Hu et al. (2024, MiniCPM) and *Hägele et al.* (2024, "Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations") study a schedule with a warm-up, a long constant-learning-rate plateau, and a short decay at the end. MiniCPM report that a decay of about 10% of the total tokens is sufficient to reach the best results, with the loss after decay equal to or below that of a cosine schedule of the same length. The useful property is that the plateau is horizon-free: a short decay branched off the constant run at any point reproduces what a matched-horizon run would have reached, at the cost of the branch alone. One constant run plus a few decay branches then gives a whole row of valid (N, D) points, making ladders cheap in D. Hägele et al. show this explicitly for scaling-law experiments.

What to read. Only the loss after decay is comparable across points. The plateau value sits well above it (the drop during decay can be a few percent of the loss, which is more than a decade of compute is worth under a 0.05 exponent) and must never be used as a rung.

Terms

learning-rate schedule The prescribed trajectory of the learning rate over a run: warm-up, then constant, cosine, linear or other decay.

warmup-stable-decay (WSD) A schedule with a short warm-up, a long plateau at constant learning rate, and a short final decay (about 10% of the run). Also called "constant plus cooldown" or "trapezoidal".

decay branch A copy of a WSD run forked from its plateau at some step and given only the decay phase; it yields the final loss a run of that length would have had.

plateau loss The loss during the constant-learning-rate phase, before decay; systematically above the decayed loss and not a valid data point.

matched-horizon run A run whose schedule was planned to end exactly where its loss is read.

Intuition

The last 10% of a run is where the loss "settles": the learning rate shrinks, the weights stop bouncing around the valley floor and drop into it. A point read before that settling is a point read from mid-air. WSD lets you settle a run wherever you like, which turns one expensive long run into many cheap valid points.

5.5 Slide 18: Refinement V: laws for architecture knobs

Once the recipe is fixed, the same machinery answers narrower questions: how sparse, what vocabulary, what precision.

What the slide shows Three cards (mixture of experts; vocabulary size; precision) with the paper behind each, and a note on data-mixing laws and the common pattern.

Details The method generalizes: fix everything else, vary one knob across a few scales, fit a law with the knob as an extra variable, extrapolate. Four examples.

Mixture of experts. Clark et al. (2022, "Unified Scaling Laws for Routed Language Models") fit a joint law in dense size and expert count for routed (MoE) models and found that the advantage of routing over a dense model at the same active parameters shrinks slowly with scale, while remaining large over the range studied. *Krajewski et al.* (2024, "Scaling Laws for Fine-Grained Mixture of Experts") add *granularity* (how small each expert is) as a variable and find that finer experts push the loss-per-compute frontier further, so that with the right granularity MoE beats dense at every scale they examine. The two sizes of an MoE (total and active) both enter the law; MAI-Base-2 (2.8T total, 80B active) is a current data point.

Vocabulary size. Tao et al. (2024, "Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies") treat vocabulary parameters separately from the rest, since a larger vocabulary makes tokens carry more information (fewer tokens per byte) at the cost of a bigger embedding and output layer. The compute-optimal vocabulary grows with the budget, more slowly than the non-vocabulary parameters, and they find most released models use vocabularies smaller than the fit recommends.

Precision. Kumar et al. (2024, "Scaling Laws for Precision") model a model trained or stored in P bits as having effective parameters $N_{\text{eff}} = N f(P)$ with f saturating as P grows, so that low precision acts like a smaller model; they also find that post-training quantization degrades more the more tokens the model was trained on, and their fits put the compute-optimal training precision near 7–8 bits, consistent with the move to FP8 training in Part Two.

Data mixtures. Ye et al. (2024, "Data Mixing Laws") find that the loss on each domain is a predictable function of the mixture proportions, so a mixture can be optimized from small runs rather than by training the large model on each candidate.

The caveat common to all four: each law is only trustworthy inside the recipe it was fit on. Change the tokenizer and the vocabulary law moves; change the attention design and the MoE law moves. A knob law is a local map, not a global one.

Terms

mixture of experts (MoE) A transformer whose feed-forward layers are replaced by many parallel "experts" of which a router activates a few per token; it separates total parameters (stored) from active parameters (used per token). See Part Two, slide 7.

active vs total parameters For an MoE, the parameters used for one token versus the parameters stored. Compute scales with active; memory and, in part, quality with total.

granularity Krajewski et al.'s knob: the size of each expert relative to a dense feed-forward layer; finer granularity means more, smaller experts.

vocabulary size (V) The number of distinct tokens the tokenizer can emit. Larger V means fewer, more informative tokens and a larger embedding table.

precision (P) The number of bits used to represent weights and activations during training or after it (FP32, BF16, FP8, 4-bit). Lower precision is cheaper and acts like fewer effective parameters.

post-training quantization Reducing the precision of a trained model's weights for cheaper inference, without retraining; its loss penalty grows with how long the model was trained.

data mixture The proportions in which domains (web, code, books, math, ...) are combined in the training corpus.

Intuition

Think of each knob as a new axis added to the (N, D) plane. The law tells you the shape of the loss surface along that axis at each scale, and in particular which way the optimum drifts as you scale up (bigger vocabulary, finer experts, lower precision up to a point). What it cannot tell you is what happens if you move the axes themselves by changing the recipe.

6 Beyond loss (slides 19–20)

6.1 Slide 19: From loss to capability: what scaling laws do not promise

Loss is smooth. A benchmark score is a nonlinear, often thresholded, function of it, and that is where "emergence" lives.

What the slide shows Three small panels against compute: loss falling smoothly; exact-match accuracy on a six-token answer rising late and suddenly ("looks like a jump"); per-token accuracy on the same model rising smoothly. The side note gives the mechanism: a six-token answer scored all-or-nothing has accuracy p^6 when each token is right with probability p.

Details Everything so far predicts loss. Users care about tasks, and task scores do not inherit the smoothness of loss.

Emergent abilities. Wei et al. (2022, "Emergent Abilities of Large Language Models") collected benchmarks (arithmetic, word unscrambling, multi-step reasoning, and others) on which models below some scale score at chance and models above it score well, with the transition happening over a narrow range of scale. They called such abilities emergent: not present in smaller models and not predictable by extrapolating their performance.

The mirage view. Schaeffer, Miranda and Koyejo (2023, "Are Emergent Abilities of Large Language Models a Mirage?") argue that the sharpness is a property of the metric rather than of the model. Exact-match accuracy on a multi-token answer awards credit only when every token is right. If the model's per-token accuracy p improves smoothly with scale, the probability of a k-token answer being entirely right is p^k, which stays near zero until p is close to 1 and then rises steeply: for $k = 6$, $p = 0.7$ gives 12%, $p = 0.9$ gives 53%, $p = 0.95$ gives 74%. Replace the metric by a continuous one on the same outputs (per-token accuracy, or the log-probability of the correct answer) and the curve is smooth; conversely, a thresholded metric can be imposed on any smooth capability to manufacture a jump. They show this on the arithmetic tasks of Wei et al. and on vision tasks where no emergence had been claimed.

What to take from it. The two views agree that loss and other continuous quantities stay on their laws, and disagree about which number matters. For forecasting, the resolution is procedural: predict a continuous intermediate quantity (loss, per-token accuracy, log-likelihood of the correct answer), and only then map it to the discontinuous score through a fitted nonlinearity. That is the two-stage map of the next slide. Whether a user experiences the jump as real is a separate question; a score that goes from 5% to 70% is a real change in usefulness however smooth its underlying cause.

Terms

emergent ability A capability that is absent (at-chance performance) below some scale and present above it, with a sharp transition; the term is from Wei et al. 2022.

exact match A scoring rule that gives credit only if the model's entire output equals the reference exactly; the canonical example of a thresholded metric.

thresholded metric Any score that applies a hard cut (right/wrong, all-or-nothing) to a graded underly-

ing quantity, discarding partial credit.

continuous metric A score that varies smoothly with the model's probabilities: per-token accuracy, Brier score, log-likelihood of the correct answer.

benchmark A fixed set of tasks with a scoring rule, used to compare models; most popular ones use thresholded metrics.

Intuition

Multiply six smooth curves together and you get a cliff. The cliff is real to anyone who needs all six tokens right, but it is not a sign that the model changed its nature at that scale. If you want to forecast where the cliff will be, forecast the smooth curves and multiply.

6.2 Slide 20: Predicting a benchmark score: the two-stage map

Fit compute to a continuous intermediate quantity, then that quantity to the score. Each stage is better behaved than the whole.

What the slide shows Two panels. Stage 1: negative log-likelihood per character on a task's correct answers against training compute, a power law through four small-model points, extrapolated as a dashed line to a star at 3.8×10^{25} FLOPs. Stage 2: benchmark accuracy against that NLL, a sigmoid through the same points, with the forecast star placed where the extrapolated NLL meets the sigmoid. The sketch follows Llama 3.

Details *Llama 3 (2024, "The Llama 3 Herd of Models", Section 3.2.1).* Meta ran scaling-law experiments between 6×10^{18} and 10^{22} FLOPs and used them in two ways. First, IsoFLOP curves gave the compute-optimal (N, D); extrapolated to the flagship budget of 3.8×10^{25} FLOPs, they suggested a 402B-parameter model on 16.55T tokens, and the model actually trained was 405B on 15.6T text tokens. Second, to predict benchmark performance, they fit (stage 1) a power law between training compute and the compute-optimal model's negative log-likelihood on the correct answers of a benchmark, measured per character so that it is comparable across tokenizers; and (stage 2) a sigmoid between that log-likelihood and the benchmark's accuracy. Stage 2 does not involve compute, so older models (the Llama 2 family) could be added as extra points to pin down the curve. The composite predicted the 405B model's scores on benchmarks such as ARC Challenge from runs roughly 10^4 times cheaper.

GPT-4 (2023, "GPT-4 Technical Report", Section 3). OpenAI reports predicting GPT-4's final loss on an internal code-derived dataset by fitting a power law with an irreducible term to models trained with at most 1/10,000 of GPT-4's compute, and predicting its pass rate on a subset of HumanEval from models with at most 1/1,000 of the compute, using the mean log pass rate as the smooth intermediate quantity (the log of the pass probability behaves like a loss, and averaging it over problems keeps it continuous).

OLMo ladders. Bhagia et al. (2024, "Establishing Task Scaling Laws via Compute-Efficient Model Ladders") make the same two-step method the organizing principle of a ladder of small open models (up to about 1B parameters, at several compute multiples), and use it to predict the task accuracy of 7B and 13B OLMo models; the paper reports which tasks are predicted within a couple of points and which are noisier, and is the clearest public recipe for doing this.

Why two stages. Each stage uses the functional form appropriate to it: compute to loss is a power law (the whole lecture), and loss to accuracy is a saturating S-curve (accuracy is bounded between chance and 100%). Fitting the composite directly would require a form that is neither, and would lose the ability to borrow stage-2 points from unrelated models.

Terms

negative log-likelihood (NLL) $-\ln p$ of the correct text under the model; the loss restricted to a particular set of strings (here, benchmark answers). Per-character normalization makes it independent of the tokenizer.

sigmoid The S-shaped function $\sigma(z) = 1/(1+e^{-z})$, used to map an unbounded quantity to a probability or an accuracy between 0 and 1.

two-stage map Forecasting a task score via an intermediate continuous quantity: compute → task NLL (power law), task NLL → accuracy (sigmoid).

HumanEval A benchmark of hand-written Python programming problems scored by whether generated code passes unit tests; the pass rate is a thresholded metric, its log a smooth one.

ARC Challenge A multiple-choice science-question benchmark used as an example target in the Llama 3 scaling analysis.

OLMo The Allen Institute's fully open language-model series, whose model ladders are published with code and data.

Intuition

Break the prediction at the point where the physics changes. Compute buys bits of log-likelihood at a steady exchange rate; accuracy converts those bits into right answers through a saturating curve that depends on the task, not on scale. Fit the exchange rate on your ladder and the conversion curve on any models you have, including old ones.

7 Ladders (slides 21–23)

7.1 Slide 21: The ladder: one recipe, several sizes, one fit

A scaling ladder is a set of models that differ only in scale. Everything else is held fixed so that scale is the only variable.

What the slide shows Five blue bars of increasing height labelled N_1 to N_5 with $20N_i$ tokens each, along a log-compute axis, and a faded tall bar marked "target". Side boxes: what a rung is, what a ladder answers (forecast, compare, allocate), and current examples (MAI-Base-2's L18–L102 ladder; OLMo's model ladders).

Details A ladder is the experimental design that produces a scaling law. Its definition has two halves. *Held fixed:* the data mixture, tokenizer, architecture family, optimizer, the shape of the learning-rate schedule, the tokens-per-parameter ratio, and the evaluation. *Varied:* the scale, usually through a single scalar (the layer count, with width and heads tied to it) so that every rung is the same "kind" of model; D varies with N through the fixed ratio.

A *rung* is one run at one scale, tuned for that scale (its own learning rate and batch from a hyperparameter law or μP, its own schedule ending where its loss is read) and evaluated identically to every other rung. The set of rungs is the ladder; the fit through them is the law; the target is the run the ladder is meant to predict.

The ladder answers three questions. *Forecast:* fit loss against compute on the rungs and extrapolate to the target. *Compare:* build two ladders differing in one choice (a data mix, a tokenizer, an attention variant); the better choice is the one whose curve is lower at the target scale, which is not necessarily the one that is lower at the smallest rung. *Allocate:* train several D per rung, fit $L(N,D)$, and solve for the compute-optimal split, as Chinchilla did.

Two current examples. MAI-Base-2 (2026, Part Two): rungs L18 to L42 across two decades of compute, extrapolated two more decades to L102, with the forecast landing on the measured value. OLMo (Bhagia et al. 2024): ladders of small open models used to forecast 7B and 13B task scores.

Terms

rung One training run at one scale within a ladder, tuned and scheduled for its own size.

recipe The complete specification of a training run except its scale: data, tokenizer, architecture family, optimizer and schedule shape, evaluation. A ladder varies scale inside one recipe.

target The large run the ladder is built to predict; usually one to two decades of compute above the top rung.

compare (ladders) Deciding between two recipe variants by building a ladder for each and reading the gap at the target scale, in compute.

allocate (ladders) Using a ladder with several token counts per size to fit $L(N, D)$ and find the compute-optimal (N, D) for a budget.

Intuition

A ladder is a controlled experiment with one independent variable. Everything in the practical advice that follows is about protecting that "one": every other difference between rungs is a confound that the fit will absorb as if it were an effect of scale.

7.2 Slide 22: Designing a ladder: the choices and the usual answers

A ladder is an experiment. Decide what is fixed, what is varied, how many points, and what is measured, before any run starts.

What the slide shows A table with eight design choices, the usual answer to each, and the reason.

Details The slide's table, expanded.

1. **How to set a rung's shape.** One scalar, typically the layer count L, with width and head count tied to it ($d \propto L$, heads $\propto d$, as in MAI-Base-2's $d = \frac{256}{3}L$). This keeps the aspect ratio roughly constant so every rung is the same kind of model, and Kaplan's finding that shape has a small effect at fixed N means little is lost. Alternatives (varying width only at fixed depth) are acceptable if applied consistently.
2. **How many rungs.** Five to eight, spaced two to four times apart in compute, spanning at least two decades. A three-constant fit (E, A, α) needs several more points than constants to have any residual degrees of freedom, and the span of the rungs bounds how far the extrapolation can reasonably go.
3. **How far to extrapolate.** One to two decades past the top rung is routine; GPT-4 reports four decades on loss; MAI-Base-2 did two. Error grows with distance, so a held-out rung between the top of the ladder and the target is the single most valuable check.
4. **Tokens per rung.** Fixed tokens per parameter (20 is the Chinchilla default; higher if you intend to over-train) gives a one-variable curve $L(C)$. Several D per N cost more but return the full $L(N, D)$ and hence the allocation; WSD decay branches make this affordable (slide 17).
5. **Hyperparameters.** Re-tuned per rung, by a fitted law or μP; never copied from the smallest rung. Porian et al. showed that fixed settings alone can move the exponent from 0.5 to 0.73.
6. **Schedule.** Each rung's schedule ends where its loss is read, or use WSD with a decay branch per point. Intermediate points of a long run are not valid rungs.

7. **What to measure.** Held-out loss in nats per token for a fixed tokenizer, or bits per byte when tokenizers differ between ladders; task NLL if the goal is a benchmark forecast (slide 20). Accuracy is not a quantity to fit a law to.

8. **Repeats.** At least two seeds on the smallest rungs. Seed-to-seed variation at 50M parameters is often larger than the effect of the recipe change being tested; without repeats, a comparison at small scale is a coin flip.

Terms

aspect ratio The ratio of a transformer's width to its depth (or related shape ratios); held roughly fixed along a ladder.

degrees of freedom (fit) The number of data points minus the number of fitted constants; with few, the fit can pass through the points without testing the functional form.

held-out rung A rung trained but excluded from the fit, used to measure how far the extrapolation can be trusted.

bits per byte (BPB) Loss normalized per byte of text rather than per token: $\mathrm{BPB} = (N_{\mathrm{tok}}/N_{\mathrm{bytes}}) \cdot L/\ln 2$. Independent of the tokenizer, so comparable across vocabularies (Part Two, slide 4).

seed The random-number seed that determines initialization and data order; two runs differing only in seed measure the noise floor of the experiment.

Intuition

Design the ladder as you would design any experiment: decide the hypothesis ("this recipe is better at 10^{24} FLOPs"), the measurement (held-out loss), the controls (one variable), the sample size (rungs and seeds) and the analysis (the fit) before spending compute. The commonest failure is to run first and decide the analysis afterwards, when the only choices left are the ones that make the data look good.

7.3 Slide 23: Fitting the law and reading the forecast

Fit in log space, hold out the top rung, report the spread. A forecast without an error bar is a guess with a decimal point.

What the slide shows Loss against compute with five fitted rungs (10^{18} to 10^{22} FLOPs), a dashed extrapolation, a shaded band widening toward the target at 10^{25}, an open-circle held-out rung at 10^{23} sitting inside the band, and a star for the target. Side box: the five-step procedure and a note on reading the band.

Details The procedure.

1. **Choose the form.** $L = E + AC^{-\alpha}$ for a one-variable ladder; $E + A/N^{\alpha} + B/D^{\beta}$ when D varies independently. Do not add terms the data cannot determine.

2. **Fit in log space with a robust loss.** Minimize a Huber loss on $\log L$ with L-BFGS from a grid of starting values and keep the best. E is the least determined constant (it is pinned by the far end of the data); fit once with E free and once with E fixed to a plausible value (an entropy estimate of the text), and see whether the forecast moves.

3. **Hold out and refit.** Refit leaving each rung out in turn (leave-one-out), and once with the top rung held out entirely. The scatter of the resulting forecasts is the error bar; the held-out top rung's residual is the single best estimate of extrapolation error.

4. **Check residuals.** Plot observed minus fitted against $\log C$. A systematic bow means the form is wrong (missing floor, regime change) or a rung is mistuned; a single outlier means a bad rung.
5. **Convert loss gaps to compute ratios.** On a line of slope α in $\log(L-E)$ against $\log C$, a gap $\Delta\log(L-E)$ between two curves corresponds to a horizontal gap of $\Delta\log(L-E)/\alpha$ decades, i.e. a compute factor $10^{\Delta\log(L-E)/\alpha}$.

Reading the band. The conversion in step 5 divides by a small number. With $\alpha \approx 0.08$, a 1% error in predicted excess loss ($\Delta\log = 0.0043$) is a factor $10^{0.0043/0.08} = 1.13$ in equivalent compute; a 5% error is a factor 1.8. Loss forecasts can be precise to a percent; compute-equivalence claims built on them inherit the error magnified by $1/\alpha$. This is why the slide insists on the band: a claim of "4× more efficient" with an unstated loss error of 5% is a claim of "between 2× and 7×".

Terms

leave-one-out Refitting with each data point removed in turn; the spread of the refits estimates how much any one point drives the result.

error bar / forecast band The range of forecasts produced by plausible alternative fits (functional forms, held-out points, fixed vs free floor); reported alongside the central forecast.

extrapolation error The difference between a forecast and the value later measured; best estimated beforehand by a held-out rung.

compute equivalence Expressing a loss difference between two recipes as the factor by which one would need more compute to match the other, read horizontally off the fitted lines.

functional form The assumed equation being fit; here a power law with or without a floor, or the two-term parametric form.

Intuition

The fit is cheap and the error bar is the product. Anyone can draw a line through five points; the question a reviewer will ask is how much the line moves when you drop a point, fix the floor or change the form. If the answer is "not much", the forecast is worth something; if it is "a lot", you need a higher rung, not a better optimizer.

8 Practice (slides 24–25)

8.1 Slide 24: Worked example: a $1M scaling study

Fictional but realistic numbers. Convert dollars to FLOPs, choose the rungs, decide what each can and cannot tell you.

What the slide shows A budget conversion ($1M → 400k GPU-hours → 5.7×10^{23} FLOPs), a table of five rungs plus a holdout with their FLOPs and GPU-hours, and two boxes: what the ladder can tell you, and sanity checks.

Details *Budget to FLOPs.* At $2.50 per H100-hour, $1M buys 400,000 GPU-hours. An H100 SXM has a dense BF16 peak of 989 TFLOP/s; at 40% model FLOPs utilization, one GPU-hour delivers $989 \times 10^{12} \times 3600 \times 0.4 \approx 1.4 \times 10^{18}$ FLOPs, so the budget is about 5.7×10^{23} FLOPs, within a factor of two of Chinchilla's training compute. All three inputs (price, peak, utilization) vary by a factor of two across clouds and codebases; the method matters more than the numbers.

The rungs. With $D = 20N$ and $C = 6ND = 120N^2$:

Rung	N	D	FLOPs	GPU-hours
1	50M	1B	3×10^{17}	0.2
2	150M	3B	2.7×10^{18}	2
3	400M	8B	1.9×10^{19}	14
4	1B	20B	1.2×10^{20}	85
5	3B	60B	1.1×10^{21}	760
holdout	8B	160B	7.7×10^{21}	5,400

Rungs 1–5 span 3.6 decades of compute and cost about 860 GPU-hours together; with three recipe variants and two seeds each, about 5,200. The 8B holdout is a decade above rung 5 and costs more than all the ladders combined. The whole study is under 3% of the budget. A 30B target at 10^{23} FLOPs (600B tokens) would be about 76,000 GPU-hours, 19% of the budget, leaving room for a second attempt if the first disagrees with the forecast.

What the ladder can tell you. The loss of the 30B run, two decades past rung 5, with the holdout as the check on the extrapolation; and which of three recipe variants is best at the target, as a compute multiplier, provided their curves have separated beyond seed noise by rung 5. *What it cannot:* the optimal tokens per parameter, which needs several D per rung (roughly tripling the ladder cost, or less with WSD branches).

Sanity checks. Rungs 1 and 2 are dominated by seed noise; plot both seeds and treat their gap as the noise floor. Rung 5 is larger than the other four combined, so a failed rung 5 reduces the ladder to four points over 2.6 decades. Keep the evaluation set fixed and decontaminated against every training set from the first run to the last; a contaminated evaluation lowers the apparent floor and biases every extrapolation downward.

Terms

GPU-hour One accelerator running for one hour; the unit cloud providers bill in. Converted to FLOPs through peak throughput and utilization.

peak throughput The maximum FLOP/s a device can execute, as published by the vendor for a given precision; H100 SXM dense BF16 is 989 TFLOP/s (1,979 with structured sparsity).

model FLOPs utilization (MFU) Achieved useful FLOP/s divided by peak, counting only forward and backward model FLOPs; 30–50% is typical for well-optimized training, 21% was MAI-Base-2's average on GB200.

decontamination Removing from the training data any documents that overlap the evaluation sets, so that measured loss reflects generalization and not memorization.

noise floor The variation between runs that differ only in seed; effects smaller than it cannot be resolved without more repeats.

Intuition

The ladder is cheap and the holdout is expensive; the target is the whole budget. That ordering is the point: a few percent of the budget spent on measurement protects the ninety-odd percent spent on the target. The holdout rung is insurance, and the premium is worth paying.

8.2 Slide 25: Pitfalls: how scaling experiments go wrong

Most failed forecasts trace back to one of these. Each has shown up in a published paper at least once.

What the slide shows Six numbered cards and a line of further pitfalls.

Details

1. **Points off one long run.** Intermediate losses of a cosine run overstate what a matched-horizon run would reach, biasing the fit toward large models (slides 11, 17). Fix: one run per point, or WSD with a decay branch per point.
2. **Copying hyperparameters up the ladder.** The best learning rate shrinks and the best batch grows with scale (slide 16). A rung with the small-rung settings sits above the curve and tilts the fit. Fix: a hyperparameter law or μP.
3. **Extrapolating too far.** Two decades past the top rung is routine; four worked for GPT-4 on loss. Error grows with distance and with uncertainty in the floor. Fix: a held-out rung, and the spread of refits as the error bar.
4. **Comparing at the wrong scale.** Recipe A beats B at 50M and loses at 3B, as Banko and Brill saw with learners in 2001 (slide 5). Fix: compare curves at the target, and only call a winner once the curves have separated by more than the seed noise.
5. **Changing two things at once.** A new tokenizer changes the unit of loss (nats per token depend on how many tokens the text becomes), so a comparison in nats per token is meaningless. Fix: compare in bits per byte; change one variable per ladder.
6. **Trusting the floor.** E is the least determined constant and dominates long extrapolations. Fix: fit with and without it; if the forecasts diverge, the data do not determine E and you need a higher rung.

Also: evaluation sets contaminated by training data (the floor looks lower than it is); loss measured on a different distribution than the one the law was fit on (an exponent is specific to its data); embedding parameters counted inconsistently across rungs (slide 11); and mixing runs with different numerics or precision, which act like different effective sizes (slide 18).

Terms

confound A variable that changes along with the one under study and whose effect the analysis cannot separate from it. In a ladder, anything that is not held fixed.

unit of loss Nats per token depends on the tokenizer; nats or bits per byte does not. Comparisons across tokenizers must use the per-byte unit.

distribution shift (evaluation) Measuring loss on text drawn from a different distribution than the training or fitting data; the law's constants and exponent do not transfer.

numerics The arithmetic precision and formats used in training (BF16, FP8, block scaling); differences between rungs act as a hidden variable.

Intuition

Every pitfall is a way of letting a second variable into the experiment: the schedule, the tuning, the distance, the scale of comparison, the tokenizer, the floor, the evaluation set. The discipline of a ladder is the discipline of saying "no" to all of them, and the published disagreements of 2020–2024 are what happens when one slips through.

9 Frontier (slides 26–29)

9.1 Slide 26: The frontier: where the laws are being pushed

The 2020–22 picture still holds for dense pretraining. Four developments change what is on the axes.

What the slide shows Four cards: sparse models; data as the constraint; laws for the whole recipe; beyond the pretraining loss. And a closing line: the method does not change, the axes do.

Details *Sparse models.* Mixture-of-experts models separate parameters from compute per token, so a law needs two sizes (total and active) and an expert configuration. Clark et al. and Krajewski et al. (slide 18) give the form; MAI-Base-2 at 2.8T total and 80B active is a recent point, with 512 experts of which 8 are active per token. The questions open in 2026 are how the expert count and granularity should scale with the budget and whether the dense "20 tokens per parameter" has a sparse analogue in active parameters, total parameters, or neither.

Data as the constraint. Public text is finite (Villalobos et al.), repetition laws exist (Muennighoff), and curation and synthetic data are now part of the recipe rather than preprocessing. For ladders this means repetition must be scaled too: a rung that sees its data once while the target sees it eight times is measuring a different regime (source scaling, Part Two).

Laws for the whole recipe. Learning rate, weight decay, dropout, vocabulary, precision and mixture each have fitted laws. The ladder has become a system that outputs the full configuration of the big run, not just its loss. MAI-Base-2's hyperparameter laws (Part Two, slides 16–17) are a worked example.

Beyond the pretraining loss. Post-training (instruction tuning, preference optimization, reinforcement learning on verifiable tasks) and inference-time compute (long chains of reasoning, search, sampling) change capability without touching the pretraining law. Whether these have their own scaling laws, and how they trade off against pretraining compute, is the active question of the period; it is outside this lecture.

Terms

sparse model A model in which only a subset of parameters is used per token; mixture of experts is the main instance.

synthetic data Training text generated by a model rather than written by people; a response to the finite stock of human text, with its own open questions about quality and diversity.

curation Filtering, deduplicating and weighting the training corpus for quality; a lever on the constant A (and possibly the exponent) of the law.

post-training All training after pretraining: supervised fine-tuning on instructions, preference optimization, reinforcement learning.

inference-time compute Compute spent while answering (longer reasoning, multiple samples, search) rather than while training; a second axis of capability.

Intuition

The straight line on log axes has survived every change of axis so far: from examples to tokens, from dense to sparse parameters, from training compute to total cost. Treat each new development as a new axis to put on the plot, not as a reason to put the ruler away.

9.2 Slide 27: Case study: the MAI-Base-2 ladder (2026)

Everything in this lecture, applied once: five rungs over two decades, a forecast two decades out, and the forecast held.

What the slide shows The MAI-Base-2 ladder as five blue bars (L18, L24, L30, L36, L42) spanning 10^{22} to $10^{23.5}$ FLOPs and the orange L102 target near $10^{25.4}$, with a dashed extrapolation. Side box: what the ladder produced.

Details Microsoft AI's report *Frontier Pre-training at Microsoft AI* (October 2026; Part Two of this series) describes a ladder built exactly along the lines of slides 21–23. The rungs are indexed by layer count L, with hidden width $d = \frac{256}{3}L$ and query heads $= L$ rounded to a multiple of 16, so that $L = 102$ reproduces the final model's $d = 8{,}704$ and 96 heads; tokens per active parameter are held fixed; and each rung repeats its data as many times as the full run will (source scaling), so that regularization choices made on small rungs transfer. The final model has 2.8T total and 80B active parameters, was trained on 50T tokens (eight passes over 6.25T unique tokens), and used about 2.4×10^{25} FLOPs on 16,000 GB200 GPUs over 38 days.

What the ladder produced, in the terms of this lecture:

- *Forecast.* A fit through L18–L42, extrapolated more than 100× in compute, predicted the final held-out evaluation of L102; the measured value landed on the prediction (their Figure 2).
- *Compare.* External models were projected onto the ladder's curve to express the recipe's advantage as compute: 4–12× FLOPs-efficiency over the strongest open base model at equal bits per byte (their Figure 3). This is slide 23's step 5 applied to another lab's model, with the caveats of slide 23's band.
- *Allocate the hyperparameters.* Laws for learning rate, weight decay and dropout as functions of steps, tokens per parameter and size, fit at small scale and validated at an intermediate scale against the previous heuristics before being used (their Figure 4).
- *Scale-dependence caught.* One stability change (activation gradient clipping) gained +2% in FLOPs-efficiency at L18 and +28% at L42 (their Figure 8). A test at the smallest rung alone would have undersold it by an order of magnitude; the ladder's job was to show the trend.

Terms

MAI-Base-2 Microsoft AI's 2.8T-parameter mixture-of-experts base model (80B active), the subject of Part Two.

activation gradient clipping Clipping the gradients of activations at selected points in the network, per token, in addition to the usual global clipping of weight gradients; MAI-Base-2's stability measure (Part Two, slide 27).

FLOPs-efficiency Microsoft AI's name for compute equivalence: the ratio of compute two recipes need to reach the same held-out loss.

GB200 NVIDIA's Grace-Blackwell superchip; 2.5 PFLOP/s dense BF16 per GPU, the hardware of the MAI-Base-2 run.

Intuition

The case study is the lecture's claims tested at the largest scale available in 2026: a two-decade extrapolation that held, a compute comparison made across labs, and a design decision that only a ladder could have made correctly. It is also a reminder of the cost: the ladder was cheap relative to the run, but the run was 16,000 GPUs for 38 days, and the forecast is what made spending that defensible.

9.3 Slide 28: Exercises

Three short ones to do by hand, two that need a GPU. Answers to the first three are on the next slide.

What the slide shows Five numbered exercises: allocate, convert, repeat (by hand); build and break it (on a GPU).

Details Notes on each, with what the exercise is testing.

1. **Allocate.** Tests the rule of thumb of slide 10 and the inference-aware shift of slide 14. The first half is arithmetic; the second half asks the student to recognize that $2N \cdot 10^{14}$ dwarfs $6ND$ for any reasonable (N, D) at 10^{22} FLOPs, so the answer is qualitative: go smaller and longer. A good answer states which direction and why, not a precise optimum (which would need a loss model and a target).
2. **Convert.** Tests slide 23's step 5 and the role of the floor. The trap is to use $\Delta \log L$ instead of $\Delta \log(L - E)$; the two answers differ by a factor of three in the exponent of the compute ratio, which is the whole point of the question.
3. **Repeat.** Tests the repetition law of slide 15 directly. The student should notice that the answer is insensitive to the exact R^* for small numbers of passes and sensitive for large ones.
4. **Build (GPU).** Tests slides 21–23 end to end at a scale that fits on one GPU in a day: five rungs from about 1M to 50M parameters at 20 tokens per parameter (so 20M to 1B tokens), WSD schedules so each rung reads a decayed loss, a fit of $E + AC^{-\alpha}$, a held-out top rung, and a leave-one-out spread. A good report shows the residual plot and the band, not only the central forecast. Expect the fitted exponent to be larger than Kaplan's (small models on a small corpus have more headroom), and expect E to be poorly determined.
5. **Break it (GPU).** Tests slide 17 by reproducing the Kaplan error deliberately: one long cosine run per size, intermediate losses read off it as if they were rungs. Expect the mismatched points to lie above the matched ones at small token counts, the apparent slope to be steeper, and the held-out error to be larger and biased (the forecast overshoots). The exercise is more convincing than the slide because the student will have seen both curves.

Terms

GPU exercise An exercise requiring training runs; at the sizes given (up to 50M parameters, 1B tokens) each rung takes minutes to an hour on a single modern GPU.

Intuition

Exercises 1–3 are the three uses of a law (budget, compare, forecast under a constraint) in miniature. Exercises 4 and 5 are the ladder and its most common failure, and the pair is designed so that the student's own data show the failure.

9.4 Slide 29: Worked answers to exercises 1–3

Showing the arithmetic, since the point of the lecture is that these are back-of-envelope calculations.

What the slide shows Three boxes with the worked answers.

Details

1. Allocate. With $C = 10^{22}$ FLOPs and $C = 120N^2$: $N = \sqrt{10^{22}/120} = \sqrt{8.3 \times 10^{19}} \approx 9.1 \times 10^9$, so $N \approx 9.1$B and $D = 20N \approx 180$B tokens. Check: $6 \times 9.1 \times 10^9 \times 1.8 \times 10^{11} = 9.8 \times 10^{21}$, within rounding of 10^{22}.

With serving: the lifetime inference cost of the 9.1B model is $2N \cdot 10^{14} = 1.8 \times 10^{24}$ FLOPs, 180 times its training cost. Any model with the same loss and fewer parameters saves $2\Delta N \cdot 10^{14}$ in serving; a 3B model saves $2 \times 6.1 \times 10^9 \times 10^{14} \approx 1.2 \times 10^{24}$ FLOPs, more than the entire training budget. Reaching the 9.1B model's loss with 3B parameters requires training on more tokens, and under Hoffmann's form the required D is large (the loss valley of slide 14 is shallow but not flat), so the precise optimum depends on the loss model; the direction is unambiguous. Sardana and Frankle's framework gives the exact answer once D_{inf} and the loss form are fixed.

2. Convert. Only the excess over the floor is on the power law. With $L - E = 0.90$ for the worse recipe and 0.92 for the better one (a gap of 0.02 nats): $\Delta \log_{10}(L - E) = \log_{10}(0.92/0.90) = 0.0096$. At slope $\alpha = 0.08$ this is $0.0096/0.08 = 0.12$ decades, a compute factor of $10^{0.12} \approx 1.32$: the better recipe is worth about $1.3\times$ compute.

Using the full loss instead, with $L \approx 2.6$: $\log_{10}(2.62/2.60) = 0.0033$, and $10^{0.0033/0.08} = 1.10$. The floor decides what fraction of the loss is "in play" under the law, so an uncertain E makes compute-equivalence claims uncertain even when the loss forecast itself is tight. (For a different split of the 0.02 gap or a different E the numbers change; the ratio of the two answers does not shrink.)

3. Repeat. Eight passes means $R_D = 7$ extra passes. $D' = U_D\left[1 + R^*(1 - e^{-R_D/R^*})\right] = 5\text{T} \times \left[1 + 15.39 \times (1 - e^{-7/15.39})\right] = 5\text{T} \times [1 + 15.39 \times 0.365] = 5\text{T} \times 6.62 \approx 33\text{T}$ effective tokens, against 40T nominal: 83% of the value of 40T fresh tokens. Eight passes lose less than a fifth of their nominal value, which is why MAI-Base-2's eightfold repetition was a reasonable design and why Muennighoff et al.'s "four epochs are nearly free" understates how far repetition can be pushed at moderate cost.

Terms

back-of-envelope calculation An estimate made with one or two significant figures and the governing formula, intended to get the order of magnitude and the direction right.

Intuition

None of the three answers needed more than a formula and a logarithm; what they needed was the right formula. The skill is knowing that $6ND$ sets the budget, $\log(L - E)$ (not $\log L$) sets the compute equivalence, and R^* sets the value of a repeat.

10 Wrap-up (slides 30–31)

10.1 Slide 30: Summary: what to remember

Six statements that survive every paper in this lecture.

What the slide shows Six cards and a reading list of three papers.

Details The six statements, with the slides that support each.

1. **Loss follows power laws** in N, D and C, over many decades, with a floor; the exponent belongs to the data more than to the architecture (slides 2–3, 6–7, 13).
2. **Balance N and D.** Compute-optimal training puts roughly twenty tokens on each parameter; serving cost pushes toward smaller models trained longer (slides 9–10, 14).
3. **Everything scales.** Learning rate, batch, weight decay, vocabulary, precision and the value of repeated data each have a law (slides 15–18).
4. **A ladder is the instrument.** One recipe, several sizes, each rung tuned and scheduled for itself; fit in log space; hold out the top rung (slides 21–23).
5. **Loss first, tasks second.** Benchmarks jump because metrics threshold; a continuous intermediate makes the forecast possible (slides 19–20).
6. **Report the error bar.** Loss forecasts are precise; compute-equivalence claims amplify small errors by $1/\alpha$; say which one you are making (slides 23, 25).

If you read three papers: Kaplan et al. 2020 for the laws and the allocation argument; Hoffmann et al. 2022 for the compute-optimal correction and the three-method design; Porian et al. 2024 for how to reconcile the two, which is also the best single account of experimental hygiene. Then Muennighoff et al. 2023 for data constraints, and the Llama 3 report (Section 3.2.1) for a forecast that was made in public and then tested.

Intuition

If a student retains one sentence: *a scaling law is a controlled experiment plus a straight line, and almost every mistake is a failure of the control, not of the line.*

10.2 Slide 31: Sources

arXiv identifiers, fitted constants and quoted numbers were checked against the papers.

What the slide shows The reference list, grouped by topic.

Details An annotated version of the list, grouped as on the slide, one line per paper on what it contributes to the lecture.

Before language models.

- Cortes, Jackel, Solla, Vapnik, Denker (NIPS 1993). *Learning Curves: Asymptotic Values and Rate of Convergence*. Fit test and training curves jointly to predict the asymptote; the first "ladder".
- Banko, Brill (ACL 2001). *Scaling to Very Very Large Corpora for Natural Language Disambiguation*. Log-linear gains to a billion words; learner rankings change with data.
- Halevy, Norvig, Pereira (IEEE Intelligent Systems 2009). *The Unreasonable Effectiveness of Data*. The web-era restatement.
- Sun, Shrivastava, Singh, Gupta (ICCV 2017), arXiv:1707.02968. *Revisiting Unreasonable Effectiveness of Data in Deep Learning Era*. JFT-300M; logarithmic gains in vision.
- Hestness et al. (2017), arXiv:1712.00409. *Deep Learning Scaling is Predictable, Empirically*. Three regions; task-specific exponents; architecture shifts, data slopes.
- Amodei, Hernandez (OpenAI blog, 2018). *AI and Compute*. Doubling every 3.4 months, 2012–2018.
- Rosenfeld, Rosenfeld, Belinkov, Shavit (2019), arXiv:1909.12673. *A Constructive Prediction of the Generalization Error Across Scales*. The first joint law in data and model size.

The core results.

- Kaplan et al. (2020), arXiv:2001.08361. *Scaling Laws for Neural Language Models*. The three laws, the joint law, the 2020 allocation, the critical batch size.
- Hoffmann et al. (2022), arXiv:2203.15556. *Training Compute-Optimal Large Language Models*. Three methods, $a \approx 0.5$, Chinchilla.
- Besiroglu, Erdil, Barnett, You (2024), arXiv:2404.10102. *Chinchilla Scaling: A replication attempt*. The published approach-3 constants are inconsistent; a refit restores 20 tokens per parameter.
- Porian, Wortsman, Jitsev, Schmidt, Carmon (2024), arXiv:2406.19146. *Resolving Discrepancies in Compute-Optimal Scaling of Language Models*. Last-layer cost, warm-up, per-scale tuning.
- McCandlish, Kaplan, Amodei (2018), arXiv:1812.06162. *An Empirical Model of Large-Batch Training*. Gradient noise scale and the critical batch size.

Refinements.

- Sardana, Frankle (2023), arXiv:2401.00448. *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws*.
- Gadre et al. (2024), arXiv:2403.08540. *Language models scale reliably with over-training and on downstream tasks*. 104 models; 32× over-trained forecast; perplexity-to-error law.
- Muennighoff et al. (2023), arXiv:2305.16264. *Scaling Data-Constrained Language Models*. $R_D^* \approx 15.4$, $R_N^* \approx 5.3$; four epochs nearly free.
- Villalobos et al. (2022, rev. 2024), arXiv:2211.04325. *Will we run out of data? Limits of LLM scaling based on human-generated data*. Data stock exhausted 2026–2032.
- DeepSeek-AI (2024), arXiv:2401.02954. *DeepSeek LLM*. Learning-rate and batch laws in compute; the M FLOPs-per-token count.
- Yang et al. (2022), arXiv:2203.03466. *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*. μP and μTransfer.
- Malladi et al. (2022), arXiv:2205.10287. *On the SDEs and Scaling Rules for Adaptive Gradient Algorithms*. Square-root learning-rate scaling for Adam-type optimizers.
- Hu et al. (2024), arXiv:2404.06395. *MiniCPM*. WSD; a 10% decay suffices.
- Hägele et al. (2024), arXiv:2405.18392. *Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations*. Constant-plus-cooldown schedules for cheap scaling experiments.

Laws for architecture knobs.

- Clark et al. (2022), arXiv:2202.01169. *Unified Scaling Laws for Routed Language Models*.
- Krajewski et al. (2024), arXiv:2402.07871. *Scaling Laws for Fine-Grained Mixture of Experts*.
- Tao et al. (2024), arXiv:2407.13623. *Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies*.
- Kumar et al. (2024), arXiv:2411.04330. *Scaling Laws for Precision*.
- Ye et al. (2024), arXiv:2403.16952. *Data Mixing Laws*.

Loss to capability.

- Wei et al. (2022), arXiv:2206.07682. *Emergent Abilities of Large Language Models*.
- Schaeffer, Miranda, Koyejo (2023), arXiv:2304.15004. *Are Emergent Abilities of Large Language Models a Mirage?*
- OpenAI (2023), arXiv:2303.08774. *GPT-4 Technical Report*, Section 3.
- Llama Team (2024), arXiv:2407.21783. *The Llama 3 Herd of Models*, Section 3.2.1.
- Bhagia et al. (2024), arXiv:2412.04403. *Establishing Task Scaling Laws via Compute-Efficient Model Ladders*.

Why power laws.

- Sharma, Kaplan (2020), arXiv:2004.10802. *A Neural Scaling Law from the Dimension of the Data Manifold*. $\alpha \approx 4/d$.
- Bahri, Dyer, Kaplan, Lee, Sharma (2021), arXiv:2102.06701. *Explaining Neural Scaling Laws*. Resolution-limited and variance-limited regimes.
- Hutter (2021), arXiv:2102.04074. *Learning Curve Theory*. Zipf features give power laws.
- Michaud, Liu, Girit, Tegmark (2023), arXiv:2303.13506. *The Quantization Model of Neural Scaling*.

Case study.

- Microsoft AI (October 2026). *Frontier Pre-training at Microsoft AI*. microsoft.ai/news. Part Two of this series.

About the figures. Curves labelled with a paper in the deck are drawn from that paper's published constants (Kaplan's three laws; Hoffmann's parametric form; Muennighoff's R_D^*; DeepSeek LLM's hyperparameter laws). Curves labelled "illustrative" are sketches of a mechanism and should not be read for numbers. The $1M worked example is fictional; its unit costs are realistic for 2026 but vary by a factor of two.

A Notation

Symbol	Name	Meaning
N	parameters	number of trainable weights (non-embedding in Kaplan et al.; all in Hoffmann et al.)
D	tokens	training tokens seen, counting repeats
C	compute	training FLOPs; $C \approx 6ND$ for a dense transformer
L	loss	held-out cross-entropy, nats per token
E	floor	irreducible loss; the entropy of the text
A, B	scale constants	prefactors of the N and D terms
α, β	exponents	of the N and D terms; $\alpha_N, \alpha_D, \alpha_C$ in Kaplan's notation
a, b	allocation exponents	$N_{\text{opt}} \propto C^a$, $D_{\text{opt}} \propto C^b$, $a + b = 1$
N_c, D_c, C_c	Kaplan's constants	set the scale of the single-variable laws
B	batch size	tokens per optimizer step (context distinguishes it from the constant B)
S	steps	optimizer steps; $D = BS$
B_{crit}	critical batch size	where the steps–examples trade-off turns
η	learning rate	step-size multiplier
U_D, U_N	unique data, base parameters	in the repetition law
R_D, R_N	extra passes, excess parameters	in the repetition law
R_D^*, R_N^*	decay constants	15.39 and 5.31 (Muennighoff et al.)
D', N'	effective data, parameters	what repeats are worth
D_{inf}	lifetime inference tokens	in the inference-aware objective
M	FLOPs per token	DeepSeek LLM's exact non-embedding count
d	intrinsic dimension	of the data manifold (slide 13); also d_{model}, the hidden width, by context

B Formulas at a glance

Power law with floor. $L(x) = E + Ax^{-\alpha}$. On log-log axes $\log(L-E) = \log A - \alpha \log x$; one decade of x multiplies $L-E$ by $10^{-\alpha}$.

Compute. $C \approx 6ND$ for a dense transformer; 1 PF-day $= 8.64 \times 10^{19}$ FLOPs; one GPU-hour = peak FLOP/s $\times 3600 \times$ MFU.

Kaplan 2020.

$$L(N) = \left(\frac{N_c}{N}\right)^{0.076}, \qquad L(D) = \left(\frac{D_c}{D}\right)^{0.095}, \qquad L(C_{\min}) = \left(\frac{C_c}{C_{\min}}\right)^{0.050}$$

$$L(N,D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}$$

Allocation $N \propto C^{0.73}$, $D \propto C^{0.27}$; critical batch $B_{\text{crit}} = B_*/L^{1/\alpha_B}$ with $\alpha_B \approx 0.21$.

Chinchilla 2022.

$$L(N,D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \qquad N_{\text{opt}} = G\left(\frac{C}{6}\right)^a, \qquad D_{\text{opt}} = G^{-1}\left(\frac{C}{6}\right)^b$$

$$a = \frac{\beta}{\alpha+\beta}, \qquad b = \frac{\alpha}{\alpha+\beta}, \qquad G = \left(\frac{\alpha A}{\beta B}\right)^{1/(\alpha+\beta)}$$

Balance at the optimum: $\alpha A N^{-\alpha} = \beta B D^{-\beta}$. Rule of thumb: $N \approx \sqrt{C/120}$, $D \approx 20N$.

Inference-aware. $C_{\text{total}} = 6ND + 2N D_{\text{inf}}$.

Repetition. $D' = U_D + U_D R_D^* \left(1 - e^{-R_D/R_D^*}\right)$ with $R_D^* \approx 15.39$; $N' = U_N + U_N R_N^* \left(1 - e^{-R_N/R_N^*}\right)$ with $R_N^* \approx 5.31$.

Hyperparameters. $\eta_{\text{opt}} = 0.3118\, C^{-0.125}$ and $B_{\text{opt}} = 0.2920\, C^{0.3271}$ (DeepSeek LLM); $\eta \propto \sqrt{B}$ for Adam-type optimizers.

Loss gap to compute. factor $= 10^{\Delta \log_{10}(L-E)/\alpha}$. At $\alpha = 0.08$, a 1% error in $L-E$ is a $1.13\times$ compute error and 5% is $1.8\times$.

Thresholded metrics. exact-match accuracy on a k-token answer $\approx p^k$ for per-token accuracy p.

Index of terms

6ND, 14

activation gradient clipping, 33
active vs total parameters, 23
AdamW beta2, 15
allocate (ladders), 27
approximation error, 16
ARC Challenge, 26
aspect ratio, 28
asymptotic error (a), 6

back-of-envelope calculation, 35
batch size (B), 11
Bayes error, 9
benchmark, 25
bits per byte (BPB), 28

C-min (compute at the critical batch), 10
Chinchilla, 14
compare (ladders), 27
compute (C), 4
compute equivalence, 29
compute-efficient frontier, 11
compute-optimal, 12
confidence interval, 15
confound, 31
confusion-set disambiguation, 8
continuous metric, 25
convergence, 11
cosine schedule, 15
critical batch size, 11
cross-entropy, 5
curation, 32

data manifold, 17
data mixture, 23
data-constrained regime, 20
decade, 6
decay branch, 22
decay constant (R star), 20
decontamination, 30
Deep Speech 2 (DS2), 9
degrees of freedom (fit), 28
dense model, 14
distribution shift (evaluation), 31

effective data (D'), 20
emergent ability, 24
epoch, 20
error bar / forecast band, 29
estimation error, 16
exact match, 24
exponent (alpha), 5
extrapolation error, 29

final loss, 13
floor (E), 5
FLOPs, 4
FLOPs-efficiency, 33
forecast, 4
functional form, 29

GB200, 33
generalization error, 7
GPT-3, Gopher, 11
GPU exercise, 34
GPU-hour, 30
gradient noise scale, 11
granularity, 23

held-out rung, 28
horizon, 15
Huber loss, 16
HumanEval, 26
hyperparameter basin, 21

inference cost, 14
inference-aware scaling, 19
inference-time compute, 32
intermediate loss, 15
intrinsic dimension (d), 17
irreducible error, 5
IsoFLOP profile, 12

JFT-300M, 8

L-BFGS, 16
ladder, 4
Lagrangian / balance condition, 16
learning curve, 6
learning rate (eta), 21
learning-rate schedule, 22

leave-one-out, 29
lifetime tokens (D inf), 19
linear and square-root scaling rules, 21
local minimum, 16
log-linear, 8
log-log plot, 5
loss (L), 4
lower envelope, 12
LSTM, RHN, 9

MAI-Base-2, 33
matched-horizon run, 22
memory-based learner, 8
mixture of experts (MoE), 23
model FLOPs utilization (MFU), 30
mu P (maximal update parametrization), 21

nat, 5
negative log-likelihood (NLL), 26
noise floor, 30
non-embedding FLOPs per token (M), 21
non-embedding parameters, 10
numerics, 31

OLMo, 26
over-training, 19
overfitting (in scaling), 10

parameters (N), 4
parametric fit, 13
peak throughput, 30
perplexity, 19
PF-day, 10
plateau loss, 22
post-training, 32
post-training quantization, 23
power law, 5
power-law region, 9
precision (P), 23
pretraining, 3

quanta (of skill), 17

recipe, 27
residual, 6
resolution-limited vs variance-limited, 17
rung, 27

sample efficiency, 10
scaling law, 3
seed, 28
sigmoid, 26
small-data region, 9
source scaling, 20
sparse model, 32
synthetic data, 32

target, 27
thresholded metric, 24
tokens (D), 4
tokens per parameter (TPP), 14
top-1 error, 19
two-stage map, 26

uniform convergence, 7
unique tokens (U D), 20
unit of loss, 31

VC dimension (h), 7
vocabulary size (V), 23

warm-up, 15
warmup-stable-decay (WSD), 22
WebText2, 10
winnow, perceptron, naive Bayes, 8

zero-shot hyperparameter transfer, 21
Zipf distribution, 17